

AI Agent Canvas

Complete Reference Guide

Build intelligent AI agents with .NET 10 and Microsoft Agent Framework.
From a single standalone agent to a coordinated multi-agent ecosystem.

Table of Contents

Guide

1. Introduction
2. AI Agents
3. Features
4. Libraries
5. Building an Agent
6. Context Domains
7. MCP and RAG
8. Multi-Agent Setup
9. Architecture
10. Behavior Patterns
11. Operations and Connectors
12. Typed Output, Vision, Orchestration and MCP Server

Appendix

User Guide

Reference

AG-UI Protocol
Security and Governance
Platform Internals

1. What Is AI Agent Canvas

AI Agent Canvas is an agent development platform built on .NET 10 and Microsoft Agent Framework. It provides the runtime, orchestration layer, and pre-built capabilities to build AI agents -- from a single standalone agent to a coordinated multi-agent ecosystem.

The platform sits between your business logic and the LLM. You write the pieces that make your agent unique (persona, tools, domain knowledge), and the platform handles the rest: the agent execution loop, context injection, tool governance, streaming, state management, and multi-agent coordination.

Scales of Deployment

Scale	Description
Standalone agent	One agent, one process, one domain focus. The simplest deployment: a single agent with its own persona, tools, and context, running in one host. Good for task-specific use cases like a compliance checker or document analyst.
Multi-agent (in-process)	Multiple specialists in one process, coordinated through handoff and background delegation. Agents share a host and can transfer conversations between each other or spin off parallel work. The user talks to one agent at a time, but the system routes to whichever specialist fits.
Agent ecosystem	Multiple hosts connected via A2A protocol over HTTP, with independent deployment and security boundaries. Each host runs its own agents and exposes them as services. Agents discover each other through AgentCards and communicate across network boundaries, owned and deployed by different teams.

The same codebase supports all three. You start with one agent and add more as the problem demands it. A standalone agent can become part of a multi-agent system by registering it in the agent registry and defining handoff rules. That multi-agent system can later federate with other hosts over A2A without changing the agent code.

What the Platform Provides

The platform handles the infrastructure so you can focus on domain logic:

- **Agent execution loop** -- the reason-act-observe cycle that drives every agent interaction
- **Context injection** -- automatic assembly of persona, entity memory, user profile, guardrails, and domain knowledge into the system prompt
- **Tool governance** -- policy-based filtering and approval rules applied before any tool executes

- **Streaming** -- real-time Server-Sent Events via the AG-UI protocol, delivering text, tool calls, and state updates to the frontend as they happen
- **State management** -- persistent chat history, entity memory, and scheduled task storage backed by SQLite
- **Multi-agent coordination** -- handoff, background delegation, async messaging, group chat and Magentic runs, and workflow orchestration across agents
- **Run safety** -- a context budget that counts the prompt with the model's own tokenizer, a loop guard that gives each run an exit condition, and cost tracking on model calls
- **Unattended operation** -- schedules, event triggers, deterministic jobs, a run ledger and daily spend limits
- **Outside accounts** -- encrypted credentials, OAuth with token refresh, and connectors whose tools that reach people need approval

What You Provide

Your job is to define what makes each agent distinct:

- **Persona** -- the agent's identity, tone, expertise, and behavioral instructions
- **Tools** -- the functions the agent can call to take action in your domain
- **Domain knowledge** -- RAG sources, entity definitions, and contextual data the agent needs to reason well
- **Guardrails** -- boundaries on what the agent should and should not do
- **Skills and workflows** -- multi-step procedures the agent follows to complete complex tasks

2. What Is an AI Agent

An AI agent is software that uses an LLM to reason about a goal, decide what actions to take, execute them through tools, and iterate until the goal is met. Unlike a chatbot that generates text in response to prompts, an agent acts on your behalf -- it reads data, calls APIs, modifies systems, and makes decisions across multiple steps.

The Agent Loop

Every agent interaction follows the same cycle:

```
User message
-> Context injection (persona, memory, guardrails, domain knowledge)
-> LLM reasoning (understand the goal, plan next steps)
-> Tool calls (execute actions via functions, APIs, or external systems)
-> Tool results (observe outcomes)
-> LLM reasoning (evaluate results, decide if done or continue)
-> Response (deliver the final answer or ask for clarification)
```

This loop repeats as many times as needed. A simple question might take one pass. A complex task -- researching a topic, pulling data from three systems, formatting a report -- might take ten. The agent decides when it has enough information to respond.

In code, this loop is managed by the agent runtime. Here is a simplified version of what happens on each turn:

```
// The runtime assembles context and runs the loop
var systemPrompt = contextManager.BuildSystemPrompt(agent, user, conversation);
var messages = new List<ChatMessage> { new SystemChatMessage(systemPrompt) };
messages.AddRange(conversation.History);
messages.Add(new UserChatMessage(userInput));

// The FunctionInvokingChatClient handles the tool-call loop automatically
var response = await chatClient.GetResponseAsync(messages, options);
```

The `FunctionInvokingChatClient` from `Microsoft.Extensions.AI` intercepts tool-call responses from the LLM, executes the corresponding functions, feeds results back, and lets the LLM continue reasoning -- all within a single call.

What Makes Agents Different from LLM Wrappers

A thin LLM wrapper sends a prompt and returns the response. An agent goes further:

- **Tool use** -- agents call functions to interact with external systems. They don't just describe what should happen; they do it. A wrapper tells you the API call to make. An agent makes the call.
- **Context awareness** -- agents maintain state across turns: who the user is, what entities are in play, what happened earlier in the conversation, and what domain knowledge applies. Each turn builds on the last.
- **Multi-step reasoning** -- agents break complex goals into steps, execute them in sequence, and adapt based on intermediate results. If step 3 fails, the agent reasons about why and tries a different approach.

- **Persistence** -- agents remember across sessions. Entity memory, user preferences, and conversation history survive restarts. The agent picks up where it left off.

Use Cases

Domain	What the Agent Does
Financial services	Monitors portfolio risk, flags compliance violations, and generates regulatory reports from transaction data.
IT operations	Triages incidents from monitoring alerts, correlates logs across systems, and executes runbook steps to resolve issues.
Customer support	Resolves tickets by looking up account data, applying business rules, and taking actions like issuing refunds or updating records.
Legal and compliance	Reviews contracts against policy checklists, extracts key clauses, and flags deviations for human review.
Healthcare	Summarizes patient records, cross-references symptoms with clinical guidelines, and prepares pre-visit briefs for clinicians.
Software engineering	Reviews pull requests, generates test cases from specifications, and investigates build failures by reading logs and code.
E-commerce	Manages product catalog updates, processes return requests end-to-end, and generates pricing recommendations from sales data.

3. Features

Agent Capabilities

These features apply to every agent, whether running standalone or as part of a multi-agent system.

Feature	Description
Dynamic personas	Each agent has a persona that defines its identity, expertise, tone, and behavioral rules. Personas are injected into the system prompt at runtime and can be swapped or layered without changing code.
Persistent context	Domain-specific context (facts, rules, reference material) is loaded into the agent's system prompt on every turn. Context is defined in markdown files and assembled by the context manager.
Entity memory	Agents remember key entities (people, projects, systems, accounts) across conversations. Entities are stored in SQLite and recalled when relevant, giving the agent long-term awareness of the domain it operates in.
Guardrails	Behavioral boundaries defined in markdown that constrain what the agent will and will not do. Guardrails are injected into the system prompt alongside the persona, enforcing policy at the reasoning level.
User profiles	The agent knows who it is talking to -- name, role, preferences, and permissions. User profiles are loaded at the start of each session and shape how the agent responds.
Skills	Named, multi-step procedures the agent can execute. Skills are registered as tools and contain structured instructions the agent follows. They turn complex workflows into repeatable, reliable operations.
Workflows	Orchestrated sequences of steps involving multiple tools, decisions, and checkpoints. Workflows can be defined in code or declared in YAML for sequential, concurrent, or conditional execution.
Scheduling	Agents can schedule tasks for future or recurring execution. Scheduled tasks are persisted in SQLite and executed by a background service. A task runs an agent, or a deterministic job when it names one.

Feature	Description
MCP connections	Agents connect to external data sources and tools through the Model Context Protocol. MCP servers are connected at runtime through <code>`McpConnectionManager`</code> , or at startup from a seed, and their tools appear alongside native agent tools. Tools from a connected server pass the governance policy and tracing. The <code>`connect_mcp_server`</code> tool is blocked by default, because it takes secrets as arguments the model can see. The MCP connector is the route for an account-backed server.
RAG pipeline	Retrieval-augmented generation backed by a SQLite vector store. Documents are chunked, embedded, and stored locally. At query time, the agent retrieves relevant chunks using cosine similarity with FTS5 hybrid search, then reranks them with the model. Needs an embedding model on the Azure AI Foundry or Databricks provider.
System tools	Built-in tools to read, write and list files and to run scripts. Both work inside an allowlist, and an empty allowlist denies everything. They are subject to the same governance as custom tools.
File workspace	<code>`FileAccessProvider`</code> gives agents a sandboxed file system for reading, writing, and managing files. Agents can work with uploaded documents, generate outputs, and organize artifacts in their workspace.
Tool governance	Policy-based filtering applied before a tool executes, using a deny-overrides evaluation model. Tools named in <code>`Security:ApprovalRequiredTools`</code> are blocked until an operator removes them from the list. The defaults are <code>`system_write_file`</code> , <code>`system_run_script`</code> , <code>`connect_mcp_server`</code> and <code>`schedule_task`</code> .
Tool approval	Interactive approval flow for sensitive tool calls. Users can approve once, approve for the session ("don't ask again"), or deny. Approval decisions are cached so repeated calls to the same tool don't interrupt the flow.
Episodic memory	Agents remember past goals, outcomes, and tool usage across sessions. Episodes are stored in SQLite with automatic relevance decay and injected into the system prompt as context, giving agents the ability to learn from experience.

Feature	Description
Audit log	Every model invocation, tool call, result, and error is recorded in a SQLite-backed audit trail. Sensitive parameters are automatically redacted. Agents can query their own audit history and retrieve aggregate statistics.
Event triggers	Proactive agent engagement through scheduled (cron), file-watch, webhook and connector triggers. Events wait in a durable SQLite queue with deduplication keys, retry with backoff, and a dead-letter state, so a restart loses none.
Computer use	Browser automation via headless Chromium (Playwright). Agents can navigate pages, click elements, type text, take screenshots, and extract page content, enabling web-based research and UI testing tasks.
Evaluation	Cases live in source, not in a database. <code>`tests/AiAgentCanvas.EvalTests`</code> holds checked-in cases graded by <code>`Microsoft.Extensions.AI.Evaluation.Quality`</code> , and CI runs them so a regression fails the build. The suite skips itself when no model is configured.
Context budget	Counts each prompt with the model's own tokenizer, enforces a ceiling per component, and compacts history with a summarizer before the provider truncates it without warning.
Loop guard	Gives each run an exit condition: a tool-round cap, a token budget, an optional cost cap, repeated-action detection and stagnation detection. When one trips it withdraws the tools and asks the model to report what it has.
Cost tracking	Records tokens and estimated cost for each model call, streaming included, using the rates in <code>`Agent:Pricing`</code> .
Run ledger	One SQLite record per scheduled, triggered, delegated, job, orchestration and external run, with usage, tool calls and outcome. Viewable in the Runs tab.
Spend limits	Daily limits per agent, per trigger and in total, measured from the run ledger. A trigger event over its limit is deferred.
Jobs	Deterministic work that runs without a model, started from a schedule, a trigger or an agent tool.

Feature	Description
Typed output	Answers in a fixed JSON shape, checked against a schema and retried with the specific errors until they fit.
Vision	Tools that describe an image or extract typed data from one. Images come only from allowlisted folders and https hosts.
Connections and connectors	Encrypted credentials, OAuth with token refresh, and connectors to outside services. A tool that sends or deletes needs approval.
Authentication	Pluggable schemes (API key and JWT bearer) protect the endpoints. Off by default, with a startup warning while it is off.
Cost-aware model routing	A delegating chat client that scores request complexity and routes simple requests to an economy model while sending complex requests to the primary model. Reduces cost without sacrificing quality on hard tasks.
Reflective reasoning	A delegating chat client that injects reflection prompts after a configurable number of consecutive tool rounds, encouraging the agent to reassess its approach and correct course before continuing.
OpenTelemetry	Distributed tracing and metrics for agent operations. Every LLM call, tool invocation, and agent turn is instrumented, providing observability into what the agent did and how long it took.
Real-time streaming	Server-Sent Events via the AG-UI protocol deliver text tokens, tool calls, reasoning steps, and state updates to the frontend as they happen. The user sees the agent thinking and acting in real time.

Multi-Agent Features

These features enable coordination between multiple agents within a single host or across hosts.

Feature	Description
Agent registry	A central registry of all agents in the system, with their personas, tools, and capabilities. Agents discover each other through the registry, which the orchestrator uses to route requests to the right specialist.

Feature	Description
Handoff	The <code>handoff_to_agent</code> mechanism transfers an active conversation from one agent to another. The receiving agent gets the conversation context and continues seamlessly. Used when a request falls outside the current agent's expertise.
Background agents	Parallel delegation to agents that work independently in the background. The primary agent continues its conversation while background agents execute tasks concurrently and report results when done.
Agent messaging	Asynchronous mailbox-based communication between agents. Agents can send and receive messages without blocking, enabling coordination patterns that don't require real-time handoff.
A2A protocol	Agent-to-Agent communication over HTTP/JSON across host boundaries. Remote agents are discovered through AgentCards that describe their capabilities. Once connected, remote agents behave like local agents -- the calling agent doesn't need to know whether its peer is in-process or across the network.
Workflow orchestration	Multi-agent workflows that coordinate work across agents in sequential, concurrent, or declarative patterns. Workflows can be defined in code or in YAML, specifying which agents handle which steps, how data flows between them, and what happens on failure.
Orchestration runs	Group chat, handoff and Magentic runs with checkpoints on disk. A Magentic run stops for a person to approve its plan and survives a restart. Only a person can answer the review.
MCP server	Serves a chosen set of this host's tools to outside clients over MCP. Nothing is exposed until configuration names it.

4. Libraries

AI Agent Canvas is built on a stack of libraries that each handle a specific concern. Understanding what each one does and where it fits will help you navigate the codebase and make informed decisions about what to extend.

Microsoft Agent Framework (MAF)

The core agent runtime. MAF provides `HarnessAgent`, the base class for all agents, along with the middleware pipeline that processes each turn -- context injection, tool resolution, governance checks, and response handling. It defines how agents are registered, how they receive messages, and how multi-agent coordination works (handoff, background delegation, messaging).

The repository targets .NET 10 and MAF 1.22. Package versions live in `Directory.Packages.props`.

Key packages:

- `Microsoft.Agents.AI` -- agent abstractions and the AI processing pipeline
- `Microsoft.Agents.AI.Harness` -- `HarnessAgent`, middleware, and the agent execution loop
- `Microsoft.Agents.AI.Workflows` -- workflow orchestration: sequential, concurrent, group chat, handoff and Magentic builders, with checkpointing
- `Microsoft.Agents.AI.Workflows.Declarative` -- YAML-based workflow definitions
- `Microsoft.Agents.AI.Hosting.AGUI.AspNetCore` and `Microsoft.Agents.AI.Hosting.A2A.AspNetCore` -- the AG-UI and A2A servers
- `Microsoft.Agents.AI.Purview` -- compliance middleware

Microsoft.Extensions.AI (MEAI)

The abstraction layer between your agent code and the LLM provider. MEAI defines `IChatClient` for chat completions, `AIFunction` and `AITool` for tool definitions, and `IEmbeddingGenerator` for embeddings. The `FunctionInvokingChatClient` wraps any `IChatClient` and automatically handles the tool-call loop -- intercepting tool-call responses, executing functions, and feeding results back to the model.

`DelegatingChatClient` enables middleware patterns like logging, caching, and governance wrapping.

MEAI makes the LLM backend swappable. The platform defaults to Azure AI Foundry and also ships Databricks and Snowflake providers, selected by the `Provider` key. Any provider that implements `IChatClient` works without changing agent code. The evaluation suite uses `Microsoft.Extensions.AI.Evaluation` and its `Quality` evaluators.

Azure AI Foundry (Azure OpenAI)

The default LLM provider. `AzureAIFoundryClientFactory` creates `IChatClient` and `IEmbeddingGenerator` instances configured for Azure OpenAI endpoints. It supports both API key and managed identity authentication, selected through configuration. The factory is registered in DI and consumed by the agent runtime -- agents never talk to Azure directly.

Azure AI Foundry is the default. Setting Provider to Databricks or Snowflake selects the other provider projects, and a provider project can register its own economy and judge clients. The rest of the platform is unaffected.

AG-UI Protocol

The streaming protocol between the agent backend and the frontend. AG-UI uses Server-Sent Events (SSE) to deliver a typed event stream: text deltas, tool call starts and completions, reasoning content, state snapshot and delta updates, interrupt requests, and run lifecycle events (started, completed, error). The frontend consumes these events to render the agent's output progressively -- the user sees text appearing, tool calls executing, and state changing in real time.

The protocol is defined by a set of event types with structured payloads. The backend emits events as the agent processes each turn; the frontend subscribes and renders.

A2A Protocol (Agent-to-Agent)

Cross-host agent communication over HTTP/JSON. A2A lets agents running in different processes or on different machines collaborate as if they were local. Each host publishes an AgentCard -- a JSON document describing the agent's name, capabilities, and endpoint. Remote agents are discovered by fetching their AgentCard and registered in the local agent registry.

Once registered, a remote agent is called through the same handoff and messaging interfaces as a local agent. The calling agent does not need to know whether its peer is in-process or across the network. A2A handles serialization, transport, and error propagation.

Model Context Protocol (MCP)

A standard protocol for connecting agents to external data sources and tools. MCP servers expose tools (functions the agent can call) and resources (data the agent can read) over a defined interface. The platform uses the `ModelContextProtocol C# SDK (1.4)` in three ways:

- **Client through the connection manager.** `McpConnectionManager` connects to a server at runtime or at startup from a seed, over HTTP or SSE, and reconnects after a failed health ping. Tools it registers at runtime join the dynamic tool registry and reach agents on their next call. They pass the governance policy and the tracing wrapper like tools registered at startup. The `connect_mcp_server` tool is blocked by default, because its secret arguments are visible to the model.
- **Client through a connector.** The MCP connector runs a client over a stored connection's guarded HTTP client, so credentials, host restrictions and OAuth refresh come from the credential store. Tools from connectors are wrapped for governance and tracing, and tools that send or delete need approval. Gmail uses this route.
- **Server.** The `McpServer` capability serves a chosen set of this host's own tools to outside clients through `ModelContextProtocol.AspNetCore`.

Microsoft Agent Governance Toolkit

Security and compliance middleware for agent operations. The governance toolkit provides two main capabilities: prompt injection detection (scanning system instructions for attempts to manipulate the agent's behavior) and policy-based tool filtering using a deny-overrides evaluation model (if any policy denies a tool call, the call is blocked, regardless of other policies that allow it).

Governance decisions generate audit events that record what was evaluated, what the outcome was, and why. These events feed into the platform's observability pipeline for compliance reporting and incident investigation.

Microsoft Purview

Compliance middleware for content classification and data loss prevention (DLP). Purview integration adds a middleware layer that classifies content flowing through the agent -- both inputs and outputs -- against sensitivity labels and DLP policies. If content matches a restricted classification, the middleware can block, redact, or flag it before it reaches the user or an external system.

This is relevant for agents operating in regulated environments where data handling must conform to organizational or regulatory policy.

SQLite + Microsoft.Extensions.VectorData

Local persistence for agent state. SQLite (through `Microsoft.Data.Sqlite`) stores chat history, scheduled tasks, episodic memory, the audit log, the run ledger, the trigger queue, connections and orchestration runs, each in its own database file. The vector store, built on `Microsoft.Extensions.VectorData`, adds embedding-based retrieval for the RAG pipeline.

The RAG implementation uses a hybrid search strategy: cosine similarity over embeddings for semantic relevance, combined with FTS5 full-text search for keyword precision. Documents are chunked, embedded via `IEmbeddingGenerator`, and stored in SQLite. At query time, both search paths run and results are merged and ranked.

SQLite was chosen for simplicity and portability -- no external database dependency, single-file storage, and good-enough performance for the agent's access patterns.

5. Building an Agent

An agent in AI Agent Canvas is a thin set of seed classes that declare what the agent *is* and what it *knows*. You write six pieces of custom code. The platform handles everything else: runtime execution, streaming, context injection, tool routing, governance, telemetry, and more.

What You Implement

1. A Persona Seed

The persona defines the agent's identity, role, and behavioral instructions. It implements `IPersonaSeed` using the `PersonaSeed` class, which takes a name, description, and freeform instructions string.

```
services.AddSingleton<IPersonaSeed>(new PersonaSeed(
    name: "financial-analyst",
    description: "A financial analyst that uses market data tools to research stocks and
        companies",
    instructions: """
        You are a financial analyst assistant.

        You have access to these tools:
        - stock_quote: Get current price and daily change for a ticker from Yahoo Finance
        - stock_history: Get historical price data with configurable range (5d, 1mo, 3mo,
            6mo, 1y, 5y)
        - edgar_company_facts: Fetch SEC EDGAR financial data including revenue, net income,
            assets, and EPS

        When analyzing a stock:
        1. Start with stock_quote for the current price and daily movement
        2. Use stock_history to show recent trends
        3. Use edgar_company_facts for fundamental data (revenue, earnings, balance sheet)
        4. Synthesize the data into a clear summary with specific numbers

        Always cite the data source (Yahoo Finance or SEC EDGAR) and note that this is
        informational,
        not investment advice.
        """);
```

The interface is minimal:

```
public interface IPersonaSeed
{
    string Name { get; }
    string Description { get; }
    string Instructions { get; }
}
```

On first startup, the persona is saved to `agent-data/orchestrator/agent/personas/financial-analyst.md`. If that file already exists, the seed is skipped -- manual edits are preserved.

2. Tool Providers

Tools are registered in a separate data connection project, not in the agent itself. This separation means multiple agents can share the same tools without duplication.

A tool provider is a class that returns `IReadOnlyList<AITool>` using `AIFunctionFactory.Create`. Each method uses `[Description]` attributes so the LLM understands what the tool does and what each parameter means.

```
public sealed class MarketDataToolProvider
{
    private readonly IHttpClientFactory _httpClientFactory;
    private readonly ILogger<MarketDataToolProvider> _logger;

    public MarketDataToolProvider(IHttpClientFactory httpClientFactory,
        ILogger<MarketDataToolProvider> logger)
    {
        _httpClientFactory = httpClientFactory;
        _logger = logger;
    }

    public IReadOnlyList<AITool> GetTools()
    {
        return
        [
            AIFunctionFactory.Create(EdgarCompanyFactsAsync, "edgar_company_facts",
                "Fetch SEC EDGAR financial data for a company by ticker symbol"),
            AIFunctionFactory.Create(StockQuoteAsync, "stock_quote",
                "Get current stock price and daily change from Yahoo Finance"),
            AIFunctionFactory.Create(StockHistoryAsync, "stock_history",
                "Get historical price data from Yahoo Finance"),
        ];
    }

    [Description("Fetch SEC EDGAR financial data for a company by ticker symbol")]
    private async Task<string> EdgarCompanyFactsAsync(
        [Description("Stock ticker symbol (e.g. AAPL, MSFT)")] string ticker,
        CancellationToken ct)
    {
        // Implementation: call SEC EDGAR API, parse response, return JSON summary
    }

    [Description("Get current stock price and daily change from Yahoo Finance")]
    private async Task<string> StockQuoteAsync(
        [Description("Stock ticker symbol")] string ticker,
        CancellationToken ct)
    {
        // Implementation: call Yahoo Finance API
    }

    [Description("Get historical price data from Yahoo Finance")]
    private async Task<string> StockHistoryAsync(
        [Description("Stock ticker symbol")] string ticker,
        [Description("Time range: 5d, 1mo, 3mo, 6mo, 1y, 5y")] string range,
        CancellationToken ct)
    {
        // Implementation: call Yahoo Finance historical data API
    }
}
```

The tool provider is registered in DI as `IReadOnlyList<AITool>`:

```
services.AddSingleton<MarketDataToolProvider>();
services.AddSingleton<IReadOnlyList<AITool>>(sp =>
    sp.GetRequiredService<MarketDataToolProvider>().GetTools());
```

3. An Agent Tools Seed

The agent tools seed declares which tools the agent depends on by name. This creates an explicit binding between the agent and its tools that the platform validates at startup. If a declared tool is not registered, the platform logs a warning.

```
services.AddSingleton<IAgentToolsSeed>(new AgentToolsSeed(
    agentName: "financial-analyst",
    toolNames: ["stock_quote", "stock_history", "edgar_company_facts"]));
```

The interface:

```
public interface IAgentToolsSeed
{
    string AgentName { get; }
    IReadOnlyList<string> ToolNames { get; }
}
```

This is a declaration, not a restriction. The platform uses it for startup validation and tool routing. The agent can still use platform-provided tools (context management, file operations, etc.) that are registered globally.

4. Context Seeds (Optional)

Beyond the persona, you can seed additional context domains. Each domain has its own seed interface, but they all follow the same pattern: implement the interface, register it as a singleton, and the platform persists it on first startup.

Guardrail seed -- policy rules the LLM must follow:

```
services.AddSingleton<IGuardrailSeed>(new GuardrailSeed(
    name: "investment-disclaimer",
    severity: "high",
    enabled: true,
    rule: """
        Never provide specific buy, sell, or hold recommendations.
        Always include a disclaimer that the analysis is informational only
        and not investment advice.
        Do not predict future stock prices or guarantee returns.
        """));
```

Context seed -- background knowledge injected into the system prompt:

```
services.AddSingleton<IContextSeed>(new ContextSeed(
    topic: "financial-analysis-methodology",
    tags: "finance,methodology",
    type: "fact",
    content: """
        ## Financial Analysis Reference
        - **P/E Ratio**: Price / Earnings per Share. Compare within sector.
        - **Revenue Growth**: Year-over-year revenue change.
        - **EPS**: Earnings per Share. Primary driver of stock valuation.
        """));
```

The full set of optional seed interfaces:

Interface	Purpose
`IGuardrailSeed`	Behavioral constraints with severity levels
`IContextSeed`	Typed knowledge (fact, reference, decision, feedback)
`IWorkflowSeed`	Multi-step workflow definitions
`IEntitySeed`	Domain entity schemas
`ISkillSeed`	Reusable prompt templates
`IUserProfileSeed`	User role, timezone, and preferences
`IMcpConnectionSeed`	MCP servers to auto-connect at startup

You can register multiple seeds of each type. Each one becomes a markdown file in the agent's data directory.

5. DI Registration

All seeds and tool registrations go into a single `IServiceModule` implementation. This is the agent's entire public API -- one class that wires everything into the service collection.

```
using AiAgentCanvas.Abstractions;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

namespace Agent.FinancialAnalyst;

public sealed class FinancialAnalystModule : IServiceModule
{
    public string SectionName => "Agents:FinancialAnalyst";

    public void ConfigureServices(IServiceCollection services, IConfiguration configuration)
    {
        // Persona
        services.AddSingleton<IPersonaSeed>(new PersonaSeed(
            name: "financial-analyst",
            description: "A financial analyst that uses market data tools",
            instructions: "..."));

        // Context, guardrails, workflows, entities, skills, profile...
        services.AddSingleton<IContextSeed>(new ContextSeed(...));
        services.AddSingleton<IGuardrailSeed>(new GuardrailSeed(...));
        services.AddSingleton<IWorkflowSeed>(new WorkflowSeed(...));
        services.AddSingleton<IEntitySeed>(new EntitySeed(...));
        services.AddSingleton<ISkillSeed>(new SkillSeed(...));
        services.AddSingleton<IUserProfileSeed>(new UserProfileSeed(...));

        // Tool binding
        services.AddSingleton<IAgentToolsSeed>(new AgentToolsSeed(
            agentName: "financial-analyst",
            toolNames: ["stock_quote", "stock_history", "edgar_company_facts"]));
    }
}
```

SectionName points at Agents:FinancialAnalyst, the configuration section that turns this module on. If Enabled there is not true, the module never runs.

The agent project only needs two dependencies: AiAgentCanvas.Abstractions (for the seed interfaces and IServiceModule) and Microsoft.Extensions.DependencyInjection.Abstractions (for IServiceCollection). No AI framework references needed -- the agent does not own tools or the LLM client.

6. Composition Root

There is no line in Program.cs to write, and no ProjectReference from Host to add. Host never names this project at all. Instead, the agent project's build output is copied into a plugins/Agent.FinancialAnalyst/ folder next to the Host executable (a post-build target in the .csproj does this), and at startup ServiceModuleExtensions.AddServiceModules loads every plugin folder it finds through its own AssemblyLoadContext, discovers the IServiceModule implementation by reflection, and calls ConfigureServices if the config section opts in.

The two things a new agent project needs, beyond implementing IServiceModule, are:

```
<PropertyGroup>
  <GenerateDependencyFile>true</GenerateDependencyFile>
</PropertyGroup>

<Target Name="CopyToHostPluginsFolder" AfterTargets="Build">
  <ItemGroup>
    <PluginOutputFiles Include="$(TargetDir)**\*.*)" />
  </ItemGroup>
  <Copy SourceFiles="@(\PluginOutputFiles)"
        DestinationFolder="$(HostPluginsDir)$(MSBuildProjectName)\%(RecursiveDir)"
        SkipUnchangedFiles="true" />
</Target>
```

GenerateDependencyFile gives the host's plugin loader a .deps.json manifest to resolve the agent's own dependencies; the post-build target is what actually places the output where the host looks. \$(HostPluginsDir) is defined once in the repo's Directory.Build.props.

The platform resolves all IPersonaSeed, IContextSeed, and other seed services once the module runs, persists them to disk, and makes them available to the runtime agent.

What the Platform Provides

Everything outside those six pieces is handled by the platform:

- **HarnessAgent runtime** -- the MAF SDK's AsHarnessAgent creates the agent with system prompt assembly, tool routing, and conversation management
- **AG-UI streaming** -- real-time streaming of agent responses to the frontend via the Agent-User Interaction protocol
- **Context injection** -- AIContextProvider implementations inject personas, guardrails, entities, profiles, and context into the system prompt before every LLM call
- **Chat pipeline** -- all agents, including persona agents, run on the same pipeline: context budget, loop guard, cost tracking, tool deduplication, and optional routing, reflection and audit

- **Governance wrapping** -- `GovernedAIFunction` and `GovernedMcpGateway` evaluate tool calls against the policy file. Tools registered at startup are wrapped, and so are tools added at runtime, such as those from a connected MCP server or a connector.
- **Run records and limits** -- scheduled, triggered and delegated runs are recorded in the run ledger and counted against spend limits
- **File workspace** -- `FileSystemAgentFileStore` gives the agent sandboxed file read/write access
- **Background agents** -- long-running tasks execute in background agent threads
- **Tool approval** -- `ToolApprovalAgentOptions` with configurable auto-approval rules
- **Telemetry** -- `OpenTelemetry` tracing and structured logging across the full request pipeline
- **Health checks** -- readiness and liveness probes for orchestrator health
- **A2A protocol** -- Agent-to-Agent communication for multi-agent scenarios
- **Sessions and history** -- conversation persistence and session management

6. Context Domains

Context domains are the agent's knowledge system. Each domain stores a specific type of information as markdown files with YAML frontmatter. The platform injects this data into the system prompt before every LLM request, so the agent always has access to its configuration, policies, and accumulated knowledge. The agent can also manage domain data at runtime through built-in tools.

The Three-Class Pattern

Every domain follows the same structure:

- **Store** -- reads and writes markdown files from layered directories. Handles CRUD operations and file parsing via the `MarkdownFile` utility.
- **Tool provider** -- exposes the store's operations as `AITool` instances so the LLM can create, read, update, and delete domain data through conversation.
- **Context provider** -- an `AIContextProvider` that reads domain data from the store and injects it into the system prompt before each LLM call.

This pattern means data flows in both directions: the platform injects domain data *into* the LLM's context, and the LLM can modify domain data *through* tool calls. A user can say "add a guardrail that prevents sharing internal pricing" and the agent will create a guardrail file that takes effect on the next request.

The Six Domains

Domain	What It Stores	System Prompt Injection
Personas	Agent identities with name, description, and behavioral instructions	Active persona's instructions appended directly to the system prompt
Context	Typed knowledge entries: fact, reference, decision, or feedback	All entries grouped by type under a <code>### Persistent Context`</code> section
Entities	Structured records describing people, companies, projects, or other domain objects	Entity index injected under <code>`### Known Entities`</code>
Guardrails	Policy rules with severity levels (critical, warning, info) and an enabled flag	Active rules injected under <code>### Guardrails & Policies`</code>
Profiles	User-specific settings: role, timezone, and freeform preferences	Active profile injected under <code>### Active User Profile`</code>

Domain	What It Stores	System Prompt Injection
Workflows	Multi-step process definitions with descriptions and tags	Not injected into the system prompt; executed on demand when the user or agent triggers them

Each domain stores its data as markdown files with YAML frontmatter. For example, a guardrail file looks like:

```

---
name: investment-disclaimer
severity: high
enabled: true
---

Never provide specific buy, sell, or hold recommendations.
Always include a disclaimer that the analysis is informational only
and not investment advice.
```

Data Storage Structure

Agent data lives under a common agent -data/ root. Each agent gets its own subdirectory, with a shared/ directory for cross-agent data. Within each subdirectory, agent/ holds system-written files (seeds and tool-created content) and user/ holds hand-written files that the system reads but never overwrites.

```

agent-data/
├── orchestrator/
│   ├── agent/                                # System writes here (seeds + tool-created content)
│   │   ├── personas/
│   │   ├── context/
│   │   ├── workflows/
│   │   ├── entities/
│   │   ├── profiles/
│   │   └── guardrails/
│   └── user/                                # Hand-written files (read-only to system)
│       ├── personas/
│       ├── context/
│       ├── workflows/
│       ├── entities/
│       ├── profiles/
│       └── guardrails/
└── shared/                                  # Accessible to all agents
    ├── agent/
    │   ├── personas/
    │   ├── context/
    │   └── ...
    └── user/
        ├── personas/
        ├── context/
        └── ...
```

The agent/ directory is where seed data lands on first startup and where tool-created content is saved. The user/ directory is a no-code alternative to implementing seed interfaces: drop a properly formatted markdown file into the right subdirectory and the platform picks it up automatically.

Layered Reading

Each store reads from multiple directories and merges the results. When the platform calls `ListAll()` on a domain store, it collects files from:

- The agent's own agent/ directory (system-written)
- The agent's own user/ directory (hand-written)
- The shared/agent/ directory (shared system-written)
- The shared/user/ directory (shared hand-written)

All files are merged into a single list. If two files have the same name, the agent-level file takes precedence over the shared-level file. This layering lets you define shared defaults (company-wide guardrails, common entity schemas) that individual agents can override with their own versions.

The context provider then reads this merged list and formats it into the appropriate system prompt section. The LLM sees user-provided, system-created, and shared data together, without needing to know where each piece came from.

How Domains Interact

Domains are independent but complementary. A typical agent configuration might include:

- A **persona** that defines the agent's role and references specific tools
- **Guardrails** that constrain the persona's behavior (e.g., "never share internal pricing")
- **Context** entries that provide background knowledge the persona needs
- **Entities** that define the domain objects the agent works with
- A **profile** that captures the user's preferences and role
- **Workflows** that define reusable multi-step processes the agent can execute

All of these are injected into the system prompt together, giving the LLM a complete picture of who it is, what it knows, what rules it must follow, and what structured processes it can run.

7. Expanding with MCP and RAG

Agents start with the tools and knowledge you seed at build time. MCP and RAG extend the agent at runtime -- MCP connects external tools and services without writing code, and RAG enriches every response with relevant documents from a vector store.

Model Context Protocol (MCP)

What MCP Does

MCP is an open standard for connecting AI agents to external tools and data sources at runtime. Instead of writing a tool provider class and redeploying, you point the agent at an MCP server endpoint and it discovers the available tools automatically. The tools appear alongside built-in tools and the LLM can call them like any other function.

Connecting via Agent Tools

The agent exposes three MCP management tools through the `McpConnectionManager`:

```
connect_mcp_server(name, endpoint, transport, bearerToken, apiKey)  -- connect and discover
tools
disconnect_mcp_server(name)                                         -- disconnect and
remove tools
list_mcp_connections()                                              -- list active
connections
```

`connect_mcp_server` is on the default `Security:ApprovalRequiredTools` list, so governance blocks it until an operator removes it from that list. The default exists because the `bearerToken` and `apiKey` arguments are visible to the model, so a secret typed into the chat passes through it, and because a connection to an outside server widens what the agent can do.

Once an operator has allowed it, a user can connect through conversation:

"Connect to the GitHub MCP server at <https://mcp.github.com/sse>"

The agent calls `connect_mcp_server` with the name, endpoint and transport type. For an account-backed server, prefer a connection and the MCP connector (see Connectors below).

How Connection Works

The `McpConnectionManager` handles the full connection lifecycle:

- The LLM calls `connect_mcp_server` with a name, endpoint URL, and transport type
- The manager creates an HTTP client transport. The transport types are `http` and `sse`. `Stdio` is not supported.
- It connects via `McpClient.CreateAsync()` and calls `ListToolsAsync()` to discover the server's tools
- Discovered tools are registered into the `DynamicToolRegistry` under the key `mcp:{name}`

- A health ping runs every two minutes. A failed ping triggers a reconnect, and the tools are registered again

Agents see the registry's tools on their next call. The default agent and each persona agent add them to their own tools, limited by the agent's tool seed when it has one. A tool removed from the registry stops being offered.

```
private async Task<string> ConnectMcpServer(string name, string endpoint,
    string transport, CancellationToken ct)
{
    IClientTransport clientTransport = transport.ToLowerInvariant() switch
    {
        "http" or "sse" => new HttpClientTransport(
            new HttpClientTransportOptions { Endpoint = new Uri(endpoint) }),
        _ => throw new ArgumentException($"Unsupported transport: {transport}"),
    };

    var client = await McpClient.CreateAsync(clientTransport, cancellationTokens: ct);
    var mcpTools = await client.ListToolsAsync(cancellationTokens: ct);

    // Register discovered tools into the dynamic tool registry
    _dynamicToolRegistry.Register($"mcp:{name}", mcpTools);

    return $"Connected to '{name}' -- {mcpTools.Count} tools discovered.";
}
```

Once connected, MCP tools appear alongside built-in tools with no distinction from the LLM's perspective. The agent can call them, and they show up in tool listings.

Governance

The GovernedMcpGateway evaluates a tool call against the policy file and the approval-required list. It applies to each tool the host wraps: tools registered at startup, tools a connected server contributes, and the `connect_mcp_server` call itself. The default policy has a rule that denies `connect_mcp_server` when the endpoint argument points at a private or internal address, which blocks the connection before it is made:

```
public McpGatewayDecision Evaluate(string agentId, string toolName, string? payload = null)
{
    var request = new McpGatewayRequest
    {
        AgentId = agentId,
        ToolName = toolName,
        Payload = payload ?? "",
    };

    var decision = _gateway.ProcessRequest(request);

    if (!decision.Allowed)
    {
        _logger.LogWarning(
            "[GOVERNANCE:MCP] Blocked tool={Tool} agent={Agent} status={Status}",
            toolName, agentId, decision.Status);
    }

    return decision;
}
```

Tools that an MCP server contributes at runtime are wrapped when they are registered, in the same governance wrapper and tracing as a tool registered at startup. A guardrail policy sees their calls, and the approval-required list applies to their names. Tools from a connector are wrapped by the connector host, which also puts any tool with risk Send or Destructive behind an approval requirement.

Building an MCP Connection Seed

To auto-connect to an MCP server at startup (instead of requiring the user to connect manually), implement `IMcpConnectionSeed`:

```
services.AddSingleton<IMcpConnectionSeed>(new McpConnectionSeed(  
    name: "company-data",  
    endpoint: "https://mcp.internal.example.com/sse",  
    transport: "sse",  
    apiKey: "key-from-your-secret-store"));
```

The interface:

```
public interface IMcpConnectionSeed  
{  
    string Name { get; }  
    string Endpoint { get; }  
    string Transport { get; }           // "sse" or "http"  
    string? BearerToken { get; }  
    string? ApiKey { get; }  
    string? ExpectedIssuer { get; }    // reject a server that names itself differently  
    IDictionary<string, string>? AdditionalHeaders { get; }  
}
```

When the `Mcp` flag is on, the platform resolves all `IMcpConnectionSeed` services at startup and connects to each one. A seed that fails to connect is logged and skipped. Seeds bypass the approval list, because the operator wrote them. Keep secrets out of source: read them from configuration or a secret store.

Connectors

A connection stores an account's credentials encrypted, and a connector turns that account into tools. The MCP connector connects to an MCP server over the connection's guarded HTTP client, so the credential comes from the store, the token refreshes, and calls reach only the server's own host. Gmail is this connector with Google OAuth. See [Operations and Connectors](#).

Serving Your Own Tools Over MCP

The reverse direction is the `McpServer` capability. It serves a chosen set of this host's tools to outside clients, with approval-gated tools left out and each call recorded. See [Typed Output](#), [Vision](#), [Orchestration](#) and [MCP Server](#).

Retrieval-Augmented Generation (RAG)

What RAG Does

RAG retrieves relevant document chunks at query time and injects them into the agent's context. Instead of fine-tuning a model on your documents or cramming everything into the system prompt, RAG searches a vector

store for the chunks most relevant to the user's current question and includes only those. The agent gets grounded, up-to-date information with source citations.

The Pipeline

Documents flow through an ingestion pipeline that prepares them for search, and queries flow through a retrieval pipeline that finds and ranks the best matches:



Chunking

The DocumentChunker recursively splits documents into chunks suitable for embedding. It follows a semantic hierarchy rather than using fixed-size character windows:

- **Split by paragraphs** (double newlines)
- If a paragraph exceeds ChunkSize, **split by sentences** (period, exclamation, or question mark followed by space or newline)
- Apply **overlap** between consecutive chunks for context continuity
- Discard chunks under 20 characters

```
public sealed class DocumentChunker
{
    public int ChunkSize { get; init; } = 512;
    public int ChunkOverlap { get; init; } = 64;

    public List<DocumentChunk> Chunk(string text, string? source = null)
    {
        // 1. Split by paragraphs
        // 2. If paragraph > ChunkSize, split by sentences
        // 3. Apply overlap between consecutive chunks
        // 4. Filter out chunks < 20 chars
    }
}
```

Property	Default	Description
`ChunkSize`	512	Maximum characters per chunk
`ChunkOverlap`	64	Characters of overlap between consecutive chunks

Embedding

Each chunk is embedded into a vector using the provider's embedding model: an Azure AI Foundry deployment or a Databricks model. Snowflake embeddings are not wired. The platform uses `IEmbeddingGenerator` from `Microsoft.Extensions.AI`, so the embedding provider is swappable via DI. The same embedding model is used for both document ingestion and query-time embedding to ensure consistent vector space alignment.

Storage

Chunks are stored as `DocumentRecord` entries in a SQLite database with two indexing strategies:

- **Vector BLOB** -- the embedding vector stored as a binary blob for cosine similarity search
- **FTS5 full-text index** -- the chunk text indexed with SQLite's FTS5 extension for keyword/BM25 search

This dual storage enables hybrid search without requiring a separate search engine.

Hybrid Search

The retrieval pipeline combines two search signals:

- **Vector search** (cosine similarity) -- captures semantic meaning. A query about "company revenue" matches chunks discussing "annual income" or "top-line growth" even without exact keyword overlap.
- **Keyword search** (SQLite FTS5 / BM25) -- captures exact term matches. Important for proper nouns, ticker symbols, product names, and technical terms that semantic search alone might miss.

The scores from both signals are combined to produce a unified ranking. This hybrid approach consistently outperforms either signal alone.

LLM Reranking

After hybrid search returns the top candidates (typically 10), the platform sends them to the LLM for relevance reranking. The LLM evaluates each candidate against the original query and assigns a relevance score. Only the top-K survivors (typically 3) are kept for injection.

This step is expensive (it costs an additional LLM call) but significantly improves precision. The initial retrieval stage optimizes for recall -- casting a wide net -- while reranking optimizes for precision -- keeping only the truly relevant chunks.

Context Injection

Surviving chunks are formatted with citation numbers and injected into the agent's context:

```
[1] source: quarterly-report-q3.pdf, score: 0.92
Revenue grew 15% year-over-year to $2.1B, driven primarily by cloud services...
```

```
[2] source: earnings-call-transcript.pdf, score: 0.87
The CEO noted that enterprise customer count increased by 23% in Q3...
```

The agent can reference these citations in its response, giving users a clear trail back to the source documents.

Enabling RAG

RAG needs two things: the Rag feature flag and an embedding model on the active provider.

```
{
  "Features": { "Rag": true },
  "AIFoundry": {
    "EmbeddingDeploymentName": "text-embedding-ada-002"
  }
}
```

For the Databricks provider, set `Databricks:EmbeddingModelName` instead. The Host registers the RAG components only when the flag is on and the active provider (Azure AI Foundry or Databricks) has its embedding setting. With the flag on and no embedding model, nothing is registered and the agent runs without retrieval. The components are:

- `DocumentChunker` for splitting documents
- `IEmbeddingGenerator` for the provider's embedding model
- A SQLite vector store with FTS5 indexing
- The hybrid search and reranking pipeline
- A RAG context provider that injects retrieved chunks before each model call

8. Multi-Agent Setup

AI Agent Canvas supports three topologies for running multiple agents. Each offers different trade-offs between simplicity, isolation, and scalability. You can combine them -- use in-process handoff and background delegation within a single host, then connect hosts with A2A when you need independent deployment.

Option 1: In-Process Handoff

All agents run in the same .NET process. The main agent delegates to a specialist by calling the `handoff_to_agent` tool. The specialist runs to completion and returns its response directly.

Under the hood, the flow is:

- `HandoffToolProvider` exposes the `handoff_to_agent` tool to the LLM
- `InProcessAgentHandoff` receives the call with the target agent name and context
- `AgentRegistry.Resolve` finds or builds the target agent (loading its persona and scoped tools)
- The target agent runs as a `ChatClientAgent` with its own persona instructions
- The response flows back to the calling agent as a tool result

When to use: Multiple specialists without deployment complexity. Shared LLM endpoint. Low latency between agents. All agents owned by the same team.

Setup

Register a persona seed and tool seed for each specialist agent. The `AgentRegistry` builds agents on demand from these seeds.

```
public static IServiceCollection AddFinancialAnalyst(this IServiceCollection services)
{
    // Persona seed defines the agent's identity and instructions
    services.AddSingleton<IPersonaSeed>(new PersonaSeed(
        name: "financial-analyst",
        description: "A financial analyst that uses market data tools to research stocks and generate reports.",
        instructions: "You are a financial analyst assistant. Use the available market data tools to research stocks, analyze trends, and generate investment reports."));

    // Tool seed restricts which tools this agent can see
    services.AddSingleton<IAgentToolsSeed>(new AgentToolsSeed(
        agentName: "financial-analyst",
        toolNames: ["stock_quote", "stock_history", "edgar_company_facts"]));

    return services;
}
```

The handoff tool is registered automatically by `HandoffToolProvider`. Any persona in the registry is available as a handoff target:

```
// HandoffToolProvider exposes this to the LLM
[Description("Delegate a task to another agent and get the result back.")]
private async Task<string> HandoffToAgent(string targetAgent, string context)
{

```

```

    var result = await _handoff.HandoffAsync(targetAgent, context);
    return JsonSerializer.Serialize(result);
}

```

When the LLM decides to delegate, it calls `handoff_to_agent` with the target name and a context message. The specialist runs in the same process, and the result comes back as a tool response.

Option 2: Background Delegation

The main agent spawns parallel tasks on background agents. This is asynchronous -- the agent can start multiple tasks, continue working on other things, and collect results when they finish.

Background delegation exposes five tools to the LLM:

Tool	Purpose
<code>`background_agents_start_task`</code>	Start a task on a named background agent
<code>`background_agents_wait_for_first_completion`</code>	Block until the first background task finishes
<code>`background_agents_get_task_results`</code>	Get the result of a specific completed task
<code>`background_agents_get_all_tasks`</code>	List all background tasks and their status
<code>`background_agents_continue_task`</code>	Send a follow-up message to a running task

When to use: Independent parallel tasks (research multiple topics simultaneously). Synthesizing results from multiple specialists. Tasks where the LLM decides at runtime which agents to involve.

Setup

Background delegation is automatic. Any persona registered in `AgentRegistry` is available as a background agent. The platform resolves all non-default agents and passes them to the `HarnessAgent`:

```

var backgroundAgents = registry.ListAvailableAgents()
    .Where(n => !n.Equals("default", StringComparison.OrdinalIgnoreCase))
    .Select(n => registry.Resolve(n))
    .Where(a => a is not null)
    .ToList();

AIAgent agent = chatClient.AsHarnessAgent(new HarnessAgentOptions
{
    Name = options.AgentName,
    BackgroundAgents = backgroundAgents is { Count: > 0 } ? backgroundAgents : null,
    // ...
});

```

No additional configuration needed. Register persona and tool seeds, and the agent becomes available for both handoff and background delegation. Persona agents run on the same chat pipeline as the default agent, so the context budget, loop guard and cost tracking apply to delegated work.

Example: Parallel Research

The LLM might use background delegation like this during a conversation:

User: Compare the financial health of Apple, Microsoft, and Google.

Agent (internal reasoning):

```
-> background_agents_start_task("financial-analyst", "Analyze Apple's financial health
    using AAPL data")
-> background_agents_start_task("financial-analyst", "Analyze Microsoft's financial health
    using MSFT data")
-> background_agents_start_task("financial-analyst", "Analyze Google's financial health
    using GOOGL data")
-> background_agents_wait_for_first_completion()
-> (collects all three results)
-> Synthesizes a comparative report
```

Option 3: A2A Protocol

Separate hosts communicating over HTTP. Each host exposes an /a2a endpoint and publishes AgentCard metadata describing its capabilities. Remote agents behave like local ones -- handoff and background tools work transparently.

When to use: Independent deployment and scaling per agent. Different security boundaries. Different teams owning different agents. Third-party agents from external organizations.

Setup

Each host runs its own instance of AI Agent Canvas with its own agents. The host exposes the A2A endpoint:

```
// In Program.cs of the remote agent host
var app = builder.Build();
app.UseAiAgentCanvas();
app.MapA2AEndpoints(agentName: "ComplianceAgent", path: "/a2a");
app.Run();
```

The A2A server is registered in DI:

```
services.AddA2AServer(agentName);
```

The orchestrator registers remote agents by URL. They join the registry and become available through the same handoff and background tools:

```
// In the orchestrator's startup
var registry = app.Services.GetRequiredService<AgentRegistry>();
registry.RegisterRemote(
    name: "compliance-checker",
    a2aUrl: "https://compliance-service.internal/a2a",
    description: "Reviews documents for regulatory compliance");
```

The RegisterRemote method creates an AgentCard, configures an HttpClient with the remote URL, and builds an agent proxy:

```
public void RegisterRemote(string name, string a2aUrl, string? description = null)
{
    var card = new AgentCard
    {
        Name = name,
        Description = description ?? $"Remote A2A agent at {a2aUrl}",
        Version = "1.0"
    };
}
```

```

var httpClient = _httpClientFactory?.CreateClient(name) ?? new HttpClient();
httpClient.BaseAddress = new Uri(a2aUrl);
var agent = card.AsAIAgent(httpClient, loggerFactory: _loggerFactory);
_agents[name] = agent;
}

```

After registration, `handoff_to_agent("compliance-checker", ...)` sends the request over HTTP to the remote host. The calling agent does not need to know whether the target is local or remote.

Option 4: Orchestration Runs

The first three options let the model decide when to involve another agent. An orchestration run starts several named agents on one task from a tool call, an API call or a person. Three kinds ship: group chat (agents take turns), handoff (a lead passes work to specialists and takes it back) and Magentic (a manager plans, assigns work and replans). Each step is checkpointed to disk, and a Magentic run can stop for a person to approve its plan, then continue after a restart. Orchestrations need `InterAgentCommunication` and the `AgentOrchestration` flag. See [Typed Output](#), [Vision](#), [Orchestration](#) and [MCP Server](#).

Comparison

	In-Process Handoff	Background Delegation	A2A Protocol
Latency	Lowest (in-memory)	Low (in-memory, async overhead)	Higher (HTTP round-trip)
Isolation	None (shared process)	None (shared process)	Full (separate hosts)
Parallelism	Sequential only	Parallel tasks	Parallel via separate hosts
Scaling	Single process	Single process	Independent per host
Complexity	Low	Low	Moderate (deployment, networking)
Security boundary	Shared	Shared	Independent (separate auth, policies)

Composing Topologies

These options are not mutually exclusive. A practical setup might look like:

- **In-process + background** for your team's agents: a financial analyst, a report generator, and a data researcher all running in one host, using handoff for sequential work and background delegation for parallel tasks.
- **A2A** for other teams' agents: the compliance team runs their own host with their own deployment schedule and security policies. Your orchestrator connects to it via `RegisterRemote`.

The agent code does not change between topologies. A persona that works as an in-process handoff target works identically as a remote A2A agent. The difference is in how you register it.

9. Architecture

The solution is organized into layers under `src/<Layer>/<Project>/`, and the Visual Studio solution folders mirror that tree one to one. References run downward. A project may depend on a layer below it and never on one above, which keeps the dependency graph clean: you can change a capability without touching the platform, or swap a data connection without affecting agent logic.



Architecture overview

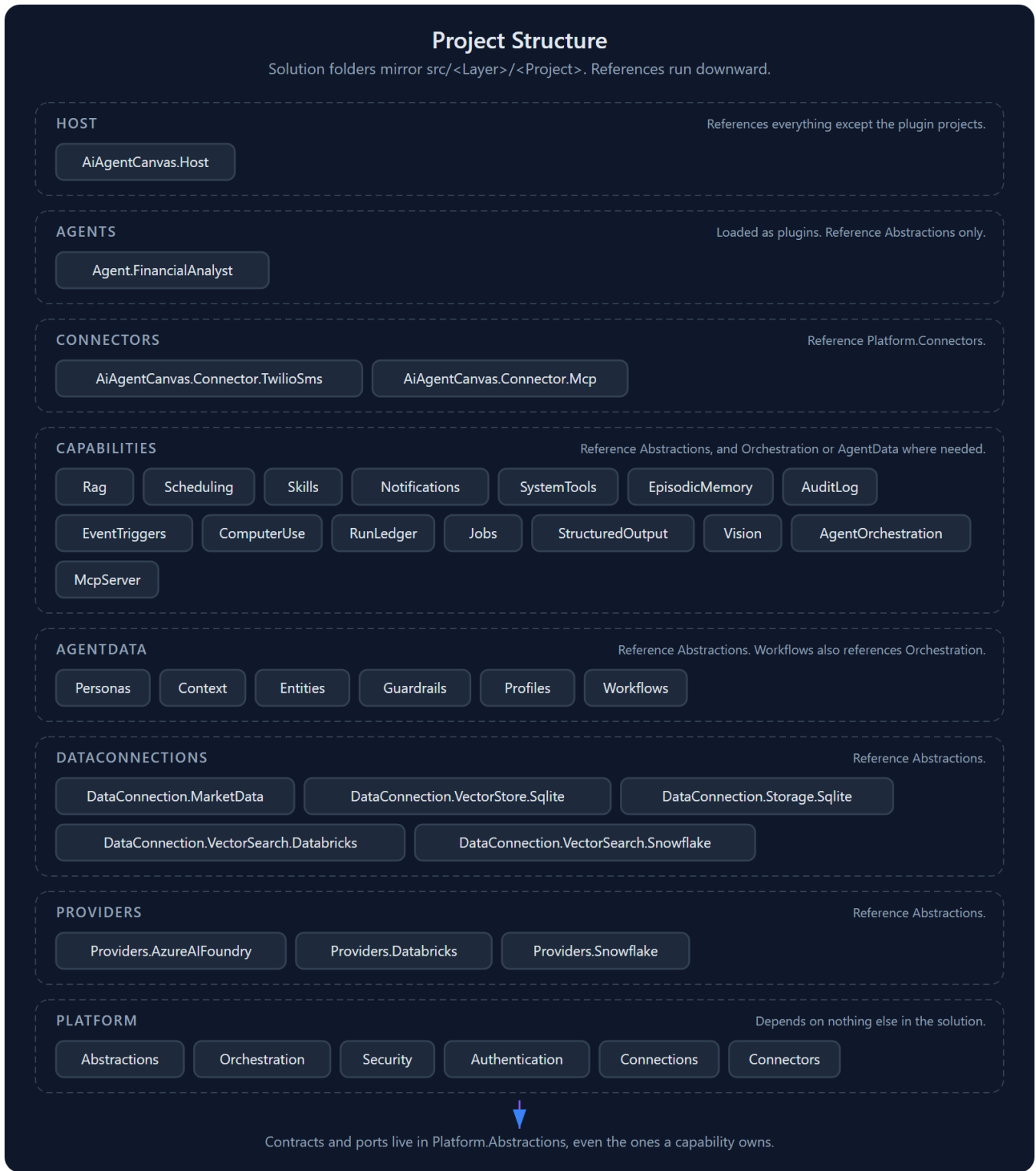
Layer Overview

Host ■■■> Capabilities, AgentData, Connectors, DataConnections, Providers, Platform
Capabilities ■■■> AgentData, Platform
AgentData, Connectors, DataConnections, Providers, Agents ■■■> Platform
Platform ■■■> nothing else in the solution

Agents are loaded as plugins at runtime, so the Host holds no compile-time reference to them.

Layer	What It Does
Host	ASP.NET Core composition root. Binds the `Features` flags, wires the layers together, and maps the HTTP endpoints with their authorization. The only project that references most of the others.
Agents	Custom agent projects. Each implements `IServiceModule` to self-register its persona, tool names and seeds. They reference only `AiAgentCanvas.Abstractions`. The host discovers them in the `plugins/` folder.
Connectors	One project per outside service: `Connector.TwilioSms` and `Connector.Mcp`, which Gmail uses. Each reference `Platform.Connectors`.
Platform	The engine and cross-cutting concerns: shared contracts, the agent runtime and chat pipeline, governance, authentication, the credential store and the connector host.
Capabilities	Opt-in feature modules, each behind a `Features:*` flag.
AgentData	Six context domains that store and provide agent knowledge: personas, context, entities, guardrails, profiles and workflows.
DataConnections	Tool providers and storage adapters: SQLite stores, the vector store, vector search and market data.
Providers	The LLM backend, chosen by the `Provider` key: Azure AI Foundry, Databricks or Snowflake.

Ports belong in `AiAgentCanvas.Abstractions`, including the ones a capability owns, such as `IAgentMessaging`, `IScheduledTaskStore`, `IRunLedger`, `ICredentialProvider` and `IStructuredResponder`. A capability that defined an interface in its own project would force its storage adapter to depend upward on the capability, which breaks the layering.



Project structure

Project Reference Map

Each project and what it references, taken from the project files.

Layer	Project	References
Platform	`Abstractions`	none
Platform	`Orchestration`	Abstractions
Platform	`Security`	Abstractions
Platform	`Authentication`	Abstractions
Platform	`Connections`	Abstractions
Platform	`Connectors`	Abstractions, Connections, Orchestration
Capabilities	`AuditLog`, `ComputerUse`, `EpisodicMemory`, `EventTriggers`, `Jobs`, `Notifications`, `Rag`, `RunLedger`, `StructuredOutput`, `SystemTools`, `Vision`	Abstractions
Capabilities	`Skills`, `AgentOrchestration`, `McpServer`	Abstractions, Orchestration
Capabilities	`Scheduling`	Abstractions, AgentData.Personas
AgentData	`Personas`, `Context`, `Entities`, `Guardrails`, `Profiles`	Abstractions
AgentData	`Workflows`	Abstractions, Orchestration
Connectors	`Connector.TwilioSms`, `Connector.Mcp`	Platform.Connectors
DataConnections	`MarketData`, `Storage.Sqlite`, `VectorStore.Sqlite`, `VectorSearch.Databricks`, `VectorSearch.Snowflake`	Abstractions
Providers	`AzureAIFoundry`, `Databricks`, `Snowflake`	Abstractions
Agents	`Agent.FinancialAnalyst`	Abstractions

The Host references all the projects above except `Agent.FinancialAnalyst` and `DataConnection.MarketData`, which are plugins loaded from the `plugins/` folder (see Service Modules below).

Projects in Detail

Platform Layer

Project	Purpose	Key Types
Abstractions	Interfaces and contracts. All other projects reference this.	<code>`IServiceModule`</code> , <code>`IAgentRegistry`</code> , <code>`IAgentHandoff`</code> , <code>`IAgentMessaging`</code> , the seed interfaces (<code>`IPersonaSeed`</code> , <code>`IAgentToolsSeed`</code> , <code>`IContextSeed`</code> , <code>`IWorkflowSeed`</code> , <code>`IEntitySeed`</code> , <code>`IGuardrailSeed`</code> , <code>`ISkillSeed`</code> , <code>`IUserProfileSeed`</code> , <code>`IGoalSeed`</code> , <code>`IMcpConnectionSeed`</code>), <code>`IRunLedger`</code> , <code>`RunTracking`</code> , <code>`AgentRunContext`</code> , <code>`IBudgetGuard`</code> , <code>`IAgentJob`</code> , <code>`ICredentialProvider`</code> , <code>`IConnectorEventSink`</code> , <code>`IStructuredResponder`</code> , <code>`ToolStateMapping`</code>
Orchestration	Agent runtime and coordination. Builds agents from seeds, runs the AG-UI and A2A servers, and owns the chat pipeline.	<code>`AgentRegistry`</code> , <code>`AgentPipeline`</code> , <code>`InProcessAgentHandoff`</code> , <code>`InProcessAgentMessaging`</code> , <code>`HandoffToolProvider`</code> , <code>`ContextBudgetChatClient`</code> , <code>`LoopGuardChatClient`</code> , <code>`CostTrackingChatClient`</code> , <code>`BudgetGuard`</code> , <code>`CoreServiceExtensions`</code>
Security	Governance and policy enforcement. Wraps tool calls with policy checks and injects security context.	<code>`GovernedAIFunction`</code> , <code>`GovernanceToolWrapper`</code> , <code>`GovernedMcpGateway`</code> , <code>`GovernanceContextProvider`</code>
Authentication	Pluggable endpoint authentication. Schemes are picked by name from <code>`Authentication:Schemes`</code> .	<code>`IAgentAuthenticationScheme`</code> , <code>`ApiKeyScheme`</code> , <code>`RequireAgentAuthorization`</code>
Connections	Encrypted credential store, the OAuth authorization code flow with PKCE, and token refresh.	<code>`ConnectionStore`</code> , <code>`CredentialProvider`</code> , <code>`OAuthService`</code> , <code>`ConnectionEndpoints`</code>
Connectors	Connector contracts, the host that runs one connector per stored connection, the guarded HTTP client, and the webhook route.	<code>`IConnectorDefinition`</code> , <code>`IToolConnector`</code> , <code>`IEventSourceConnector`</code> , <code>`ConnectorHost`</code> , <code>`ConnectorTools`</code>

The Chat Pipeline

The model is wrapped in a chain of delegating chat clients before an agent sees it. AgentPipeline builds the chain, and the default agent and the persona agents use it. From the outside in:

LoopGuard ■ Reflection ■ Audit ■ ModelRouter ■ ContextBudget ■ CostTracking ■ ToolDedupe ■ provider

Client	Job
`LoopGuardChatClient`	Ends a run that has used its tool rounds, token budget or cost cap, or that repeats itself or stops making progress. It withdraws the tools and asks the model to report what it has.
`ReflectiveChatClient`	Optional. Injects a reflection prompt after a number of consecutive tool rounds.
`AuditingChatClient`	Present when `AuditLog` is on. Records each model call.
`CostAwareModelRouter`	Optional. Sends simple turns to the economy model.
`ContextBudgetChatClient`	Counts the prompt with the model's tokenizer, enforces a ceiling per component and compacts history with a summarizer.
`CostTrackingChatClient`	Records tokens and estimated cost for each call, streaming included.
`ToolDeduplicatingChatClient`	Removes duplicate tool definitions.

Tools get the same treatment from AgentPipeline.WrapTools: the governance wrapper first, then TracedAIFunction, which records a span and the call in the ambient run record.

AgentData Layer

Six domain-specific projects that store and provide agent knowledge. Each follows the same pattern: a store for persistence, tools for LLM access, and a context provider for automatic injection.

Domain	Store	Tools	Context Provider
Personas	`PersonaStore`	`create_persona`, `update_persona`, `list_personas`, `read_personas`, `switch_personas`, `delete_personas`	`PersonaContextProvider` injects the active persona

Domain	Store	Tools	Context Provider
Context	<code>`PersistentContextStore`</code>	<code>`save_context`</code> , <code>`update_context`</code> , <code>`list_context`</code> , <code>`read_context`</code> , <code>`delete_context`</code>	<code>`PersistentContextProvider`</code> injects all stored entries
Entities	<code>`EntityStore`</code>	<code>`save_entity`</code> , <code>`update_entity`</code> , <code>`read_entity`</code> , <code>`search_entities`</code> , <code>`list_entities`</code> , <code>`delete_entity`</code>	<code>`EntityContextProvider`</code> injects the entity index
Guardrails	<code>`GuardrailStore`</code>	<code>`create_guardrail`</code> , <code>`update_guardrail`</code> , <code>`list_guardrails`</code> , <code>`toggle_guardrail`</code> , <code>`delete_guardrail`</code>	<code>`GuardrailContextProvider`</code> injects enabled rules
Profiles	<code>`UserProfileStore`</code>	<code>`create_user_profile`</code> , <code>`update_user_profile`</code> , <code>`switch_user_profile`</code> , <code>`read_user_profile`</code> , <code>`list_user_profiles`</code> , <code>`delete_user_profile`</code>	<code>`UserProfileContextProvider`</code> injects the active profile
Workflows	<code>`WorkflowStore`</code>	<code>`create_workflow`</code> , <code>`list_workflows`</code> , <code>`read_workflow`</code> , <code>`run_workflow`</code> , <code>`run_sequential_workflow`</code> , <code>`run_concurrent_workflow`</code> , <code>`delete_workflow`</code> , plus <code>`list_declarative_workflows`</code> and <code>`run_declarative_workflow`</code>	None. Workflows run on request.

Capabilities Layer

Features that agents use but that are not specific to any single domain. Each is registered only when its flag is on.

Capability	What It Provides	Key Types
Skills	Reusable prompt-template procedures, a skill registry, runtime authoring and the MCP connection manager	<code>`SkillStore`</code> , <code>`SkillToolProvider`</code> , <code>`SkillAuthoringToolProvider`</code> , <code>`LocalSkillRegistry`</code> , <code>`McpConnectionManager`</code>

Capability	What It Provides	Key Types
Scheduling	Cron-based and one-time scheduled tasks that run an agent or a job	`ScheduledTaskRunner`, `ScheduledAgentJob`, `SchedulerToolProvider`
Notifications	Agent-to-user notification delivery over SSE	`InMemoryNotificationSink`, `NotificationEndpoint`
SystemTools	File read, write and list and script execution inside allowlists	`SystemToolProvider`, `SystemToolOptions`
RAG	Chunking, hybrid retrieval, LLM reranking and cited context	`DocumentChunker`, `LlmReranker`, `RagContextProvider`
EpisodicMemory	Cross-session memory with relevance decay, search and context injection	`EpisodicMemoryStore`, `EpisodicMemoryToolProvider`, `EpisodicMemoryContextProvider`, `MemoryDecayService`
AuditLog	Model call and tool call audit trail with sensitive parameter redaction	`AuditLogStore`, `AuditingChatClient`, `AuditLogToolProvider`
EventTriggers	Scheduled, file-watch, webhook and connector triggers held in a durable queue	`TriggerRegistry`, `TriggerStore`, `TriggerEventQueue`, `TriggerDispatchService`, `ConnectorEventBridge`
ComputerUse	Browser automation through headless Chromium (Playwright)	`BrowserSession`, `ComputerUseToolProvider`
RunLedger	One record per unattended run, with usage, tool calls and outcome	`SqliteRunLedger`, `RunLedgerEndpoints`, `RunLedgerToolProvider`
Jobs	Deterministic work that runs without a model	`JobRunner`, `JobToolProvider`
StructuredOutput	Schema-checked answers with retry	`StructuredResponder`, `JsonSchemaValidator`, `SchemaCatalog`
Vision	Image tools with allowlisted sources	`ImageLoader`, `VisionToolProvider`
AgentOrchestration	Group chat, handoff and Magentic runs with checkpoints and human sign-off	`OrchestrationRunner`, `OrchestrationStore`, `OrchestrationEndpoints`
McpServer	Serves chosen tools over MCP	`ExposedToolSet`, `McpServerExtensions`

Connections and connectors sit in the Platform layer because other projects depend on them. Platform.Connections stores credentials and runs OAuth. Platform.Connectors defines the connector contracts and runs one connector per stored connection. See Operations and Connectors.

DataConnections and Providers

Storage backends and external data sources. These projects implement the persistence and data-access interfaces defined in Abstractions.

DataConnections:

Project	Purpose	Key Types
Storage.Sqlite	SQLite store for scheduled tasks	`SqliteScheduledTaskStore`, `SqliteStoreBase`
VectorStore.Sqlite	SQLite vector store for RAG embeddings, and the chat history provider	`SqliteVectorStore`, `SqliteChatHistoryProvider`
MarketData	Stock quotes, price history and SEC EDGAR company facts. A plugin.	`MarketDataToolProvider`
VectorSearch.Databricks	Databricks Vector Search index queries for grounding	`DatabricksVectorSearchToolProvider`
VectorSearch.Snowflake	Snowflake Cortex Search queries for grounding	`SnowflakeCortexSearchToolProvider`

Providers:

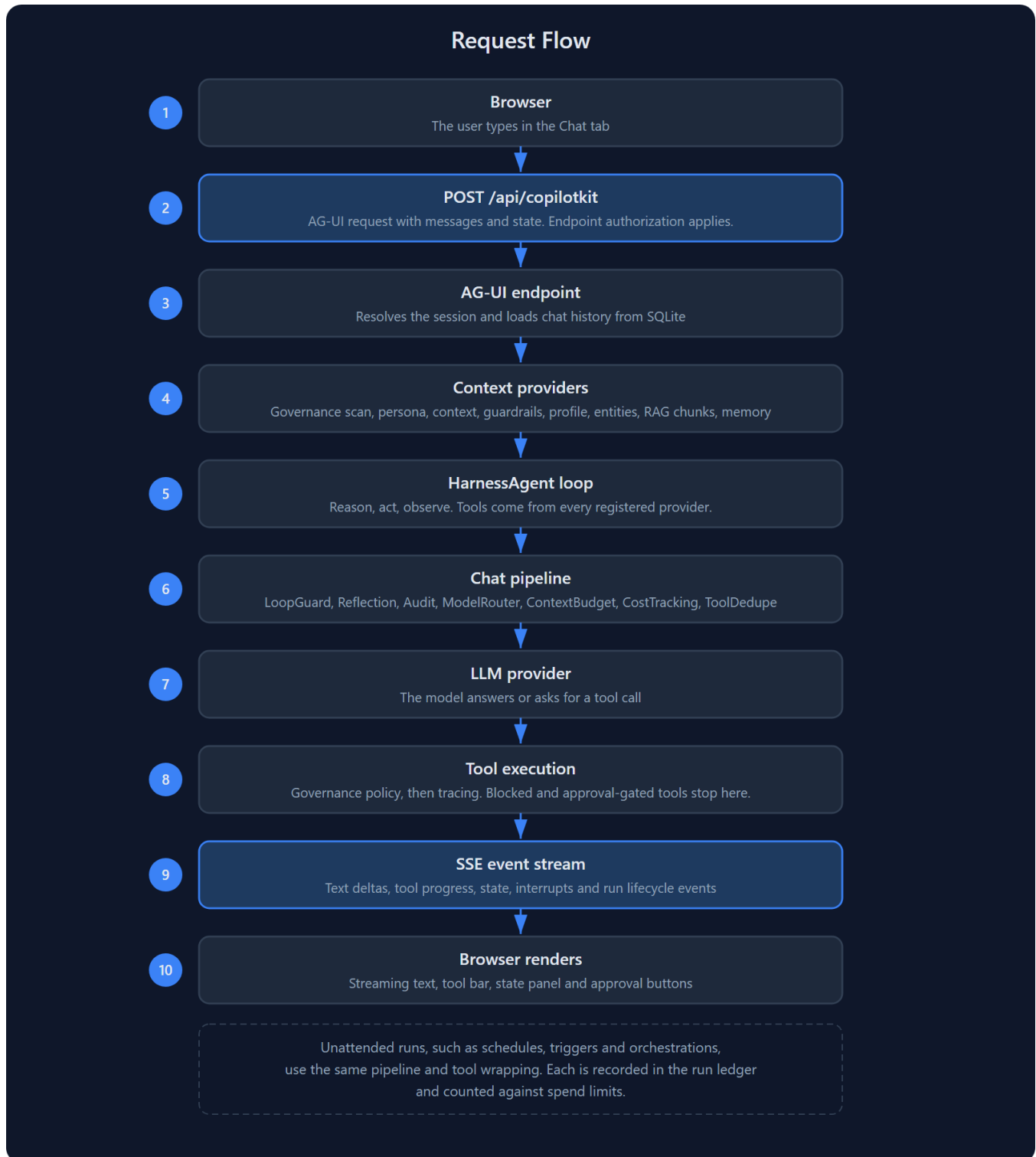
Project	Purpose
Providers.AzureAIFoundry	LLM client for Azure AI Foundry (Azure OpenAI) endpoints
Providers.Databricks	LLM client for Databricks Foundation Model APIs (OpenAI-compatible)
Providers.Snowflake	LLM client for Snowflake Cortex

Each provider registers its chat client and, where configured, embeddings plus keyed economy and judge clients.

Frontend

The frontend is a Next.js 15 application using React 19. It connects to the backend through the AG-UI protocol, a raw SSE (Server-Sent Events) client that consumes the event stream directly. There is no CopilotKit SDK dependency. The client handles event types such as TEXT_MESSAGE_CONTENT, TOOL_CALL_START, TOOL_CALL_END, STATE_SNAPSHOT and STATE_DELTA to render the chat, tool activity and live state.

The page has four tabs. **Chat** is the conversation. **Runs** lists the run ledger with totals and run detail. **Orchestrations** shows multi-agent runs and is where a person answers a plan review. **Connections** adds, checks and removes connected accounts. In development, `npm run dev` forwards `/api` requests to the backend on `http://localhost:5149`. A production build is a static export that the Host serves from `wwwroot`.

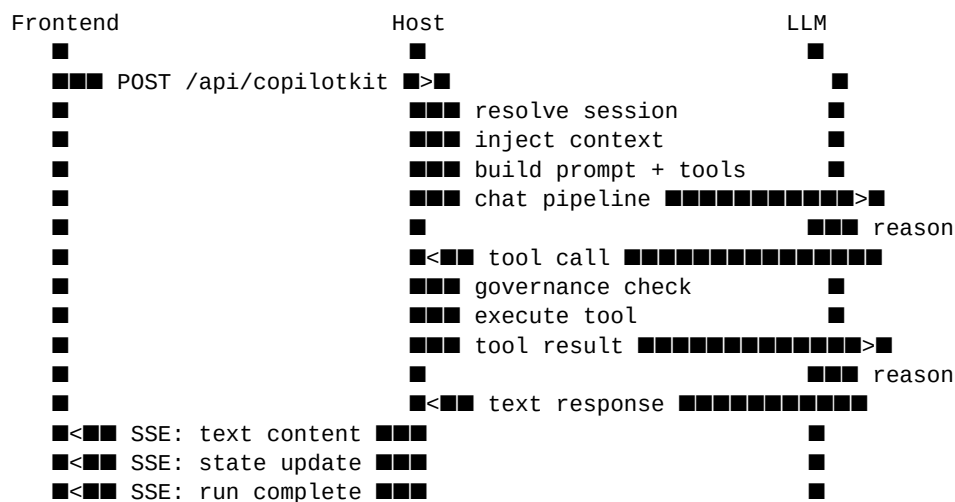


Request flow

Request Flow

A user message travels through the system in this sequence:

- **Frontend POSTs to /api/copilotkit**, the AG-UI endpoint. The request carries the user message, conversation history and frontend state. Endpoint authorization applies when authentication is on.
- **AG-UI server resolves the session**, identifies the user, loads conversation history and prepares the agent context.
- **Context providers inject data**. Each registered AIContextProvider appends its knowledge to the system prompt: the governance scan, the default prompt, persona instructions, context entries, guardrail rules, the user profile, the entity index, and RAG results and episodic memory when those capabilities are on.
- **HarnessAgent runs the agent loop**. The agent built through AsHarnessAgent executes the reason-act-observe cycle. Each model call goes through the chat pipeline described above.
- **Tool calls pass the governance wrapper and tracing**. The GovernedMcpGateway evaluates a call against the policy file and the approval-required list. A blocked call returns an error message and does not execute. Each decision is logged as an audit event.
- **Results stream as AG-UI SSE events**. Text content, tool call progress, state updates, interrupts and completion signals reach the frontend as they happen.



Runs that start without a person, such as scheduled tasks, trigger events, job runs and orchestrations, use the same pipeline and tool wrapping. Each is recorded in the run ledger and counted against the spend limits.

Feature Flags and Service Modules

The platform uses two mechanisms to control which capabilities are active at runtime.

Feature Flags

Capabilities are controlled by boolean flags in the Features section of `appsettings.json`. All flags default to false, so the platform is opt-in. When a flag is true, the capability's services, tools and endpoints are registered. The flags are read once at startup, and there is no runtime toggle.

```
{
  "Features": {
    "Personas": true,
    "Skills": true
  }
}
```

The `FeatureFlags` class in the `Host` project binds to this section. `Program.cs` checks each flag before calling the capability's registration method. A few flags have prerequisites that the `Host` checks at startup: `Connectors` needs `Connections`, `AgentOrchestration` needs `InterAgentCommunication`, daily budgets need `RunLedger`, and `McpServer` needs authentication on.

Service Modules

Agent projects and data-connection projects use the `IServiceModule` interface (defined in `Platform.Abstractions`) instead of feature flags. Each module declares a configuration section name and a `ConfigureServices` method.

Unlike every other layer, `Host` does not hold a project reference to these modules at all. They are loaded at runtime from a `plugins/` folder next to the `Host` executable, one subfolder per plugin, using `System.Runtime.Loader.AssemblyLoadContext.ServiceModuleExtensions.AddServiceModules` walks that folder, loads each plugin's assembly into its own `PluginLoadContext`, reflects over its types for `IServiceModule` implementations, and calls `ConfigureServices` for each one whose config section explicitly sets `Enabled` to `true`.

`PluginLoadContext` resolves a plugin's own private dependencies from its folder via `AssemblyDependencyResolver`, but defers to whatever the host already has loaded for anything shared, most importantly `AiAgentCanvas.Abstractions` itself. Without that, the plugin's copy of `IServiceModule` would be a distinct runtime type from the host's copy, and the `is IServiceModule` check would fail even though the source is identical.

This means adding a new agent or data-connection project touches nothing in `Host`: no `Program.cs` change, no `ProjectReference`, not even a comment. Two things make a project loadable this way:

- Implement `IServiceModule` against `Abstractions`.
- Add a post-build target that copies the project's own build output into `$(HostPluginsDir)$(MSBuildProjectName)\` (see `Directory.Build.props` for `HostPluginsDir`, and any existing plugin project, such as `Agent.FinancialAnalyst.csproj`, for the target itself). Also set `<GenerateDependencyFile>true</GenerateDependencyFile>` so `AssemblyDependencyResolver` has a manifest to resolve the plugin's own dependencies.

The project still needs to be part of the solution so a normal solution build produces its output, but `Host`'s own `.csproj` never names it.

```
Agents:FinancialAnalyst:Enabled = false (default, not registered)
DataConnections:MarketData:Enabled = true (opted in)
```

Provider-Specific Configuration

Provider-specific settings are nested under their provider section rather than in `Features`. The `Databricks Vector Search` tool, for example, lives at `Databricks:VectorSearch` and self-gates based on whether its required settings are present, so it needs no explicit `Enabled` flag.

10. Agent Behavior Patterns

Agent behavior patterns describe how an agent reasons, decides, and interacts. The platform does not enforce a pattern -- the pattern emerges from how you write the persona instructions, which tools you expose, and how you wire up inter-agent communication. The same platform primitives support all seven patterns.

Overview

Pattern	Key Mechanism	When to Use
Sequential	Persona instructions with ordered steps	Linear analysis pipelines, ETL, report generation
Reactive (ReAct)	Tool loop (built-in)	Open-ended tasks where the agent decides what to do next
Parallel (Fan-out/Fan-in)	Background agents	Independent sub-tasks that can run concurrently
Deliberate (Planning)	Workflow definitions	Complex multi-step tasks with predefined procedures
Reflective (Self-Critique)	Critic agent or self-review step	High-stakes outputs needing quality verification
Hierarchical (Delegation)	Handoff to worker agents	Complex problems with clear domain boundaries
Collaborative (Multi-Agent Debate)	Agent messaging	Decisions benefiting from diverse viewpoints

1. Sequential

The agent follows a fixed sequence of steps defined in its persona instructions. Each step runs to completion before the next begins. The LLM does not decide the order -- the instructions dictate it.

When to use: Predictable multi-step workflows where the order is known upfront. Report generation, data extraction pipelines, onboarding checklists.

How to implement: Define the steps in the persona seed's instructions. The agent's tool loop follows them in order.

```
services.AddSingleton<IPersonaSeed>(new PersonaSeed(  
    name: "daily-report-generator",  
    description: "Generates daily market reports following a fixed procedure.",  
    instructions: ""  
    You generate a daily market report. Follow these steps in order:  
  
    1. Use `stock_quote` to get the current price for AAPL, MSFT, and GOOGL  
    2. Use `stock_history` to get the 5-day price history for each
```

3. Use `edgar\_company\_facts` to get the latest quarterly revenue for each
4. Write a summary report with price trends and revenue comparison
5. Save the report using `save\_context` with topic "daily-report-`{date}`"

Do not skip steps. Do not reorder steps.
 """);

The LLM sees these instructions in its system prompt and executes the tool calls in the specified order.

2. Reactive (ReAct)

The default behavior pattern. The LLM reasons about the current situation, decides which tool to call, observes the result, and repeats until the goal is met. This is the standard reason-act-observe loop that every agent runs by default.

When to use: Most tasks. Any situation where the agent needs to decide at runtime what to do next based on the current state.

How to implement: No special configuration. Every agent uses this pattern by default. The `FunctionInvokingChatClient` handles the loop automatically:

```
// The reactive loop is built into the agent runtime
AIAgent agent = chatClient.AsHarnessAgent(new HarnessAgentOptions
{
    Name = "assistant",
    ChatOptions = new ChatOptions
    {
        Instructions = "You are a helpful assistant.",
        Tools = tools,
    },
});
```

The LLM receives all available tools and decides which to call based on the user's request. If the first tool call does not produce a satisfactory result, the LLM reasons about what to try next.

3. Parallel (Fan-out/Fan-in)

The main agent spawns multiple background tasks on specialist agents, waits for them to complete, and synthesizes the results. Tasks run concurrently.

When to use: Independent sub-tasks that do not depend on each other. Researching multiple topics simultaneously. Gathering data from multiple sources in parallel.

How to implement: Use background agent tools. Any persona in the `AgentRegistry` is automatically available as a background agent.

```
LLM decides to fan out:
-> background_agents_start_task("financial-analyst", "Analyze AAPL financial health")
-> background_agents_start_task("financial-analyst", "Analyze MSFT financial health")
-> background_agents_start_task("financial-analyst", "Analyze GOOGL financial health")
-> background_agents_wait_for_first_completion()
-> background_agents_get_all_tasks() // check which are done
-> background_agents_get_task_results(task_id) // collect each result
-> Synthesize comparative report from all three results
```

You can also use the workflow executor for structured parallel execution:

```
// Programmatic concurrent execution via MAF workflow builder
var agents = agentNames
    .Select(name => registry.Resolve(name))
    .Where(a => a is not null)
    .ToList();

var workflow = AgentWorkflowBuilder.BuildConcurrent(agents);
var input = new ChatMessage(ChatRole.User, userInput);
await using var run = await InProcessExecution.RunAsync(workflow, input, cancellationTokens:
    ct);
```

4. Deliberate (Planning)

Workflow definitions decompose goals into named steps. Instead of relying on the LLM to figure out the steps at runtime, you define the procedure upfront. The agent follows the workflow, using its tools to execute each step.

When to use: Complex multi-step tasks with well-known procedures. Compliance reviews, deployment checklists, data migration pipelines.

How to implement: Create workflows using the workflow tools or seed them at startup. Three execution modes are available:

Simple workflow -- a markdown-defined procedure the agent follows:

```
services.AddSingleton<IWorkflowSeed>(new WorkflowSeed(
    agentName: "compliance-reviewer",
    name: "quarterly-review",
    content: ""
    # Quarterly Compliance Review

    ## Steps
    1. Pull all transactions from the last quarter
    2. Flag any transaction over $10,000 without documentation
    3. Check each flagged transaction against the exceptions list
    4. Generate a findings report with severity ratings
    5. Create follow-up tasks for any critical findings
    ""));
```

The agent calls `run_workflow("quarterly-review", "Run the Q3 2025 review")` and the executor feeds the workflow steps into the LLM as structured instructions.

Sequential workflow -- chains multiple agents in order, each receiving the previous agent's output:

```
var workflow = AgentWorkflowBuilder.BuildSequential(agents);
await using var run = await InProcessExecution.RunAsync(workflow, inputMessage,
    cancellationTokens: ct);
```

Declarative workflow -- YAML-defined workflow graphs loaded and executed via MAF's

`DeclarativeWorkflowBuilder`:

```
var workflow = DeclarativeWorkflowBuilder.Build<ChatMessage>(filePath, options);
await using var run = await InProcessExecution.RunAsync(workflow, inputMessage,
    cancellationTokens: ct);
```

5. Reflective (Self-Critique)

The agent reviews its own output before delivering it, or hands off to a dedicated critic agent that evaluates quality. The review step catches errors, improves clarity, and verifies that the output meets requirements.

When to use: High-stakes outputs: legal documents, financial reports, customer-facing content. Any situation where getting it wrong is expensive.

How to implement: Two approaches.

Self-review via persona instructions:

```
services.AddSingleton<IPersonaSeed>(new PersonaSeed(  
    name: "report-writer",  
    description: "Writes reports with built-in quality review.",  
    instructions: ""  
    After generating any report:  
    1. Review each claim for accuracy against the source data  
    2. Check that all requested sections are present  
    3. Verify that numbers are consistent across sections  
    4. If you find errors, fix them before delivering the final version  
    5. State what you checked at the end of the report  
    ""));
```

Critic agent via handoff:

```
// Register a critic persona  
services.AddSingleton<IPersonaSeed>(new PersonaSeed(  
    name: "quality-reviewer",  
    description: "Reviews agent outputs for accuracy and completeness.",  
    instructions: ""  
    You are a quality reviewer. When given a document:  
    1. Check all factual claims against available data  
    2. Identify any logical inconsistencies  
    3. Flag missing information  
    4. Return a verdict: PASS, REVISE (with specific feedback), or FAIL (with reasons)  
    ""));  
  
// The main agent's instructions tell it to use the critic  
// "After generating the report, use handoff_to_agent('quality-reviewer', report) to get a  
    quality review."
```

6. Hierarchical (Delegation)

A manager agent decomposes a complex problem into sub-tasks and delegates each to a specialist worker agent via handoff. Workers return results. The manager synthesizes the final output.

When to use: Complex problems with clear domain boundaries. Each sub-domain has its own expertise, tools, and knowledge. The manager orchestrates but does not do the domain-specific work.

How to implement: Register specialist agents with scoped tools. The manager agent's persona instructs it to delegate:

```
// Manager agent with broad tool access  
services.AddSingleton<IPersonaSeed>(new PersonaSeed(  
    name: "project-manager",  
    description: "Orchestrates complex projects by delegating to specialists.",
```

```

instructions: ""
You are a project manager. When given a complex request:
1. Break it into sub-tasks based on domain expertise
2. Delegate each sub-task to the appropriate specialist using handoff_to_agent
3. Review each specialist's response for quality and completeness
4. Synthesize a final response that integrates all results

Available specialists:
- financial-analyst: market data, stock analysis, financial reports
- compliance-reviewer: regulatory checks, policy validation
- report-writer: document formatting, narrative writing
""));

// Each specialist has its own scoped tools
services.AddSingleton<IAgentToolsSeed>(new AgentToolsSeed(
    agentName: "financial-analyst",
    toolNames: ["stock_quote", "stock_history", "edgar_company_facts"]));

services.AddSingleton<IAgentToolsSeed>(new AgentToolsSeed(
    agentName: "compliance-reviewer",
    toolNames: ["list_guardrails", "read_guardrail", "read_entity"]));

```

The manager calls `handoff_to_agent` for each sub-task and receives structured results. It then combines them into a coherent final response.

7. Collaborative (Multi-Agent Debate)

Multiple agents share perspectives on the same problem via messaging. Different personas reason about the same question from different angles. A synthesizer agent collects the perspectives and produces a balanced conclusion.

When to use: Decisions that benefit from diverse viewpoints. Risk assessments, strategy decisions, ethical reviews. Cases where a single perspective might miss important considerations.

How to implement: Use `IAgentMessaging` for asynchronous inter-agent communication:

```

// Define agents with different perspectives
services.AddSingleton<IPersonaSeed>(new PersonaSeed(
    name: "optimist-analyst",
    description: "Analyzes opportunities and upside potential.",
    instructions: "Analyze the given scenario focusing on opportunities, growth potential,
        and best-case outcomes. Be specific about what could go right.));

services.AddSingleton<IPersonaSeed>(new PersonaSeed(
    name: "risk-analyst",
    description: "Analyzes risks and downside potential.",
    instructions: "Analyze the given scenario focusing on risks, threats, and worst-case
        outcomes. Be specific about what could go wrong.));

```

The orchestrator fans out the question to both analysts (via background delegation or handoff), collects their perspectives, and synthesizes a balanced view:

```

Orchestrator:
-> background_agents_start_task("optimist-analyst", "Evaluate expanding into the European
    market")
-> background_agents_start_task("risk-analyst", "Evaluate expanding into the European
    market")
-> Wait for both to complete

```

```
-> Synthesize: "The optimist sees X, Y, Z. The risk analyst flags A, B, C. Balanced recommendation: ..."
```

For more structured collaboration, agents can exchange messages directly:

```
public interface IAgentMessaging
{
    Task SendAsync(AgentMessage message, CancellationToken cancellationToken = default);
    Task<AgentMessage?> ReceiveAsync(string agentName, CancellationToken cancellationToken = default);
    Task<IReadOnlyList<AgentMessage>> ReceiveAllAsync(string agentName, CancellationToken cancellationToken = default);
}
```

Tool Scoping

`IAgentToolsSeed` restricts which tools each agent can see. When the `AgentRegistry` builds an agent from a persona, it filters the global tool set to include only the tools named in that agent's seed.

```
// In AgentRegistry.BuildAgentForPersona
var allTools = _toolsFactory().ToList();
var tools = _toolSeeds.TryGetValue(persona.Name, out var seed)
    ? allTools.Where(t => seed.ToolNames.Contains(t.Name)).ToList()
    : allTools;
```

If no tool seed is registered for an agent, it receives all tools.

Benefits of tool scoping:

- **Focus** -- fewer irrelevant tools means the LLM spends less time reasoning about tools it should not use. A financial analyst does not need to see scheduling tools.
- **Safety** -- limit access to dangerous or sensitive tools. A read-only analyst should not have write tools.
- **Cost** -- fewer tools in the prompt means fewer tokens in tool descriptions, which reduces LLM costs on every turn.

Register a tool seed alongside the persona seed:

```
services.AddSingleton<IAgentToolsSeed>(new AgentToolsSeed(
    agentName: "financial-analyst",
    toolNames: ["stock_quote", "stock_history", "edgar_company_facts"]));
```

Choosing a Pattern

Start with Reactive (the default) and add structure only when the task demands it.

```
Is it a single-step task?
■ ■ Yes -> Reactive (default tool loop)
■ ■ No
    ■ ■ Is the sequence of steps known upfront?
    ■ ■ ■ Yes -> Sequential (persona instructions) or Deliberate (workflow)
    ■ ■ ■ No -> Reactive
    ■ ■
    ■ ■ Are there independent sub-tasks?
    ■ ■ ■ Yes -> Parallel (background agents)
    ■ ■
    ■ ■ Does it need domain-specific specialists?
```

- ■■ Yes -> Hierarchical (manager delegates to workers)
-
- Does the output need quality verification?
- ■■ Yes -> Reflective (self-review or critic agent)
-
- Would multiple perspectives improve the decision?
- Yes -> Collaborative (multi-agent debate)

Patterns compose. A hierarchical system can use parallel fan-out to run workers concurrently. A deliberate workflow can include a reflective step at the end. A collaborative debate can feed into a sequential report-generation pipeline. Start simple and layer patterns as complexity grows.

Operations and Connectors

This guide covers two groups of features. The first group serves agents that run without a person watching: a record of each run, spend limits, durable event triggers and deterministic jobs. The second group serves agents that act inside outside accounts: stored credentials, OAuth, and connectors to services such as Twilio and Gmail.

All of it is off by default and sits behind `Features: *` flags.

Flag	Adds	Needs
<code>`RunLedger`</code>	A record of each run, the <code>`/api/runs`</code> endpoints, and the Runs tab	none
<code>`Jobs`</code>	Deterministic jobs that need no model	none
<code>`Connections`</code>	Encrypted credentials, the OAuth connect flow, token refresh	none
<code>`Connectors`</code>	Connectors, their tools, webhooks and events	<code>`Connections`</code>

`Agent:Budgets` is a configuration section and not a flag. It needs `RunLedger` and `Agent:Pricing`, and the Host refuses to start with budgets enabled and no ledger.

Run ledger

An unattended agent leaves no trace in a chat window. The run ledger writes one record per run to SQLite: what started it, what it did, what it cost and how it ended.

A record holds the agent name, the source (Interactive, Scheduled, Trigger, Handoff, Job, Orchestration or External), the trigger or task id, start and end times, status, the input and output text, token counts, estimated cost, the tool calls with outcome and duration, and the reason a run was cut short, if it was.

- **Nesting.** A run started inside another run, such as a handoff, records its parent. The parent's totals include the child's usage. Totals reported by the API count top-level runs only, so delegated work is not counted twice.
- **Restarts.** A run still marked Running when the process starts again is marked Abandoned. Its outcome is unknown.
- **Text limits.** Input and output are cut to `MaxTextChars`.
- **Scope.** Scheduled tasks, triggers, handoffs and jobs are recorded. Chat sessions through the AG-UI endpoint are not.

Setting (`Agent:RunLedger`)	Default	Meaning
`DatabasePath`	`run-ledger.db`	SQLite file
`RetentionDays`	30	Finished runs older than this are deleted daily
`MaxTextChars`	4000	Longest stored input or output

Read the ledger through the **Runs** tab, the endpoints (GET /api/runs, /api/runs/totals, /api/runs/{id}, endpoint key runs), or the agent tools list_recent_runs, get_run and run_totals. The list takes hours, agent, status, source and triggerId filters.

Spend limits

Each model call is priced from Agent:Pricing, which lists the rate per million tokens for each model. Two limits use those prices.

Per run. Agent:LoopGuard:MaxRunCost ends a run once it has spent that amount. The loop guard withdraws the tools and asks the model to report what it has. The ledger records the termination as CostBudget.

Per day. Agent:Budgets limits unattended spend over a rolling 24 hours, measured from the ledger, so it survives a restart.

```
{
  "Agent": {
    "Pricing": { "Models": { "gpt-4o": { "InputPer1M": 2.5, "OutputPer1M": 10 } } },
    "LoopGuard": { "MaxRunCost": 0.50 },
    "Budgets": {
      "Enabled": true,
      "TotalDailyLimit": 20,
      "DailyLimit": 5,
      "Agents": { "FinancialAnalyst": 10 },
      "Triggers": { "reply-to-texts": 3 }
    }
  }
}
```

A limit of zero means no limit. Before a scheduled task or trigger event starts a model run, the budget guard checks the limits for the agent, the trigger and the total.

- A **trigger event** over its limit is deferred without spending an attempt, and the guard is asked again after `DeferMinutes`.
- A **scheduled agent task** over its limit is skipped for that tick. Job tasks, which use no model, still run.
- A notification goes out once per day when a limit passes `warnAtFraction` (default 0.8) and once when it is reached.

A run that is still in flight does not count until it ends, so concurrent runs can pass a limit by their own cost. Set `MaxRunCost` to bound that overshoot.

Durable event triggers

Event triggers hold their queue in SQLite. A restart loses no pending event and a full queue refuses events and says so, where an in-memory channel dropped them without notice.

Delivery is at least once. An event interrupted mid-run runs again, so the work an agent does for an event must be safe to repeat. Send tools accept an idempotency key for this reason.

- **Deduplication.** An event carries an optional dedupe key. A second event with the same trigger and key is dropped. Scheduled triggers key by minute, file watchers key by path, change type and second, and a webhook sender sets the Idempotency-Key header.
- **Retry.** A failed run is retried with exponential backoff up to MaxAttempts. After that the event is parked as Dead and a notification goes out.
- **Inspection.** GET /api/triggers/queue shows counts. GET /api/triggers/events/dead lists parked events with their last error. POST /api/triggers/events/{id}/retry gives one a fresh set of attempts. The tools trigger_queue_status, list_failed_trigger_events and retry_trigger_event do the same for an agent.
- **Webhooks.** A body over MaxWebhookBodyBytes gets 413. A full queue gets 503 with Retry-After.

Setting (Agent:EventTriggers)	Default
`DatabasePath`	`event-triggers.db`
`QueueCapacity`	256
`MaxAttempts`	3
`RetryBaseSeconds`	30
`MaxRetryDelayMinutes`	15
`SucceededRetentionDays`	7
`MaxWebhookBodyBytes`	262144
`RunTimeoutMinutes`	10

Jobs

Some scheduled work needs no model: prune a table, post a nightly summary, check a queue. A job is a class that implements IAgentJob.

```
public sealed class NightlyReportJob : IAgentJob
{
    public string Name => "nightly-report";
    public string Description => "Posts yesterday's run totals.";

    public async Task<JobResult> RunAsync(JobContext context, CancellationToken ct)
    {
        // read arguments from context.Arguments, do the work, return a result
        return JobResult.Success("done");
    }
}
```

```
services.AddAgentJob<NightlyReportJob>();
```

Job names are lowercase letters, digits and hyphens. A job reports a problem it found by returning `JobResult.Failure`. An exception means the job itself broke. A job does not overlap itself: a second start while one is running is skipped. A job that runs longer than `Agent:Jobs:TimeoutSeconds` (default 600) is cancelled and recorded as failed. Each run appears in the ledger with source Job.

A job can be started from a scheduled task (`schedule_job`), from a trigger (`targetJob` on `create_trigger` or `create_connector_trigger`), or by an agent with `run_job`. `list_jobs` lists what is registered. Two jobs ship with the platform: `run-failure-report` summarizes recent failed runs and notifies, and `trigger-queue-health` reports parked events.

Connections

A connection is one configured account for one connector: a Gmail mailbox or a Twilio account. Enable Connections to store them.

Where secrets live. Metadata (label, status, settings) is stored in the clear in `connections.db`. Secrets are encrypted with ASP.NET Core Data Protection before they reach the database. The key ring lives in `KeyRingPath` and, on Windows, is also wrapped with DPAPI. Whoever can read both the key ring and the database can read all stored secrets, so keep the key ring out of source control and out of the web root. Back up both files together: without the key ring the stored secrets cannot be read.

No endpoint returns a secret. The connection list shows status, scopes and settings only.

Key connections. `POST /api/connections/key` stores an API key or a key pair (for Twilio, an API key SID and secret). Extra secrets, such as a webhook signing token, go in extras. The **Connections** tab has a form for this.

OAuth connections. The OAuth flow uses the authorization code grant with PKCE. The state value is encrypted and expires, so the callback completes only a flow this host started. Built-in providers are `google`, `microsoft`, `slack`, `github`, `notion` and `hubspot`. Add or override one under `Connections:OAuthProviders`.

- Register an OAuth application with the provider. Set its redirect address to `{PublicBaseUrl}/api/connections/oauth/callback`.
- Enter the client id and secret on the Connections tab, or set `Connections:OAuthApps:<provider>` in configuration.
- Choose the connector and select **Connect**.

`Connections:PublicBaseUrl` is required for OAuth. The redirect address is built from it and not from the request, so a forged Host header cannot send an authorization code elsewhere.

Refresh. An access token within `RefreshSkewSeconds` of expiry is refreshed before it is handed out. Refreshes are serialized per connection, because providers that rotate refresh tokens invalidate the old one at once and a second concurrent refresh would fail. A background sweep refreshes tokens due to expire soon. When a provider answers `invalid_grant`, the connection becomes `NeedsReauth`, its tools stop, and one notification asks a person to reconnect.

Setting (‘Connections’)	Default
<code>`DatabasePath`</code>	<code>`connections.db`</code>

Setting (`Connections`)	Default
`KeyRingPath`	`data-protection-keys`
`ProtectKeysWithDpapi`	true (Windows only)
`PublicBaseUrl`	none
`ReturnUrl`	`/`
`RefreshSkewSeconds`	120
`OAuthApps`, `OAuthProviders`	empty

The management endpoints use endpoint key connections. The OAuth callback has no authorization attribute, because the browser arrives from the provider carrying no credentials of ours.

Connectors

A connector turns a service into tools, events or documents. The full contract is in `docs/design/connectors.md`. This section describes what an operator sees.

Enable Connectors (and Connections). The Host registers the shipped connectors, runs one instance per stored connection, and reconciles every `ReconcileSeconds` (default 15), so a connection added on the Connections tab starts within seconds.

Tool names. Tools are named `{prefix}_{verb}`. The first connection of a connector uses the connector's own prefix (`twilio_sms_send`). A later connection adds its label (`twilio_sms_frontdesk_send`) so two accounts do not share names. If the first connection is removed, the next one takes the plain prefix.

Risk and approval. Each tool declares a risk: Read, Write, Send or Destructive. A tool that sends to another person or deletes is wrapped in an approval requirement. The default interactive agent shows the approval prompt in chat. Agents without an approval channel, and unattended runs, do not execute the call. Where agents must send without a person present, set `Connectors:ApprovalMode` to `Audit`. Calls are still traced and counted.

Network rules. A connector cannot build its own HTTP client. The one it receives:

- sends only to hosts in the connector's allowed list, over https (http is accepted for loopback only);
- follows no redirects;
- attaches the stored credential and, on a 401 with an OAuth token, refreshes once and retries once;
- retries only requests that are safe to repeat, so the host does not repeat a send automatically;
- reports `aiagentcanvas.connector.calls` and `aiagentcanvas.connector.duration`, and logs the host and path but not the query string or any header.

Health. The host checks each connection every `CheckIntervalSeconds` (default 300). A connection the service refuses becomes `NeedsReauth`. A connection that fails for another reason shows `Error` with the detail, and clears when a later check passes.

Webhooks and events. A connector with the Events capability receives webhooks at `/api/connectors/{connectionId}/webhook`. The route carries no endpoint authorization. The connector

checks the sender's own signature and the route refuses a failed check with 401 before parsing anything. The Connections tab shows each connection's webhook address.

A verified event goes to the trigger system. Create a trigger for it:

```
create_connector_trigger(name: "reply to texts", connectionId: "tw1",
    eventType: "sms.received", agentMessage: "Draft a reply to this text.")
```

eventType matches exactly, or by family with a trailing . * (sms.*). The event id is the dedupe key, so a service that delivers the same webhook twice runs the agent once. The sender's text reaches the agent after a line that marks it as data from an outside party. A full queue answers the webhook with 503 so the service retries.

Twilio SMS

Setting	Meaning
`account_sid`	Required. The account SID (`AC` and 32 hex digits)
`from_number`	A sender number in E.164 form. Either this or `messaging_service_sid` is required
`messaging_service_sid`	A Messaging Service SID to send from
`quiet_hours`	For example `21:00-08:00`. Sends in this window are refused
`timezone`	IANA zone for quiet hours. Default UTC
`max_sends_per_hour`	Default 30

Credentials are a key pair: an API key SID and secret, or the account SID and auth token. Webhooks need the account auth token stored as the AuthToken extra, which verifies X-Twilio-Signature.

Tools: `twilio_sms_send` (risk Send), `twilio_sms_list`, `twilio_sms_get` and `twilio_sms_missed_calls` (risk Read). Events: `sms.received`, `sms.failed` and `call.missed`.

Protections in the send tool: the number must be E.164, the body is capped at 1600 characters, a recipient who replied STOP is refused until they reply START, and the quiet-hours and hourly limits apply. A repeated idempotencyKey within 24 hours returns the first result and sends nothing. Opt-outs, the idempotency window and the hourly count are held in memory, so they reset on restart. Twilio enforces opt-outs itself, and the connector learns of one from the first refused send.

Gmail and other MCP servers

The MCP connector runs an MCP client over the connection's guarded HTTP client, so the credential comes from the store and the token refreshes. Nothing about the account passes through the model. This replaces typing a bearer token into `connect_mcp_server`.

Setting	Meaning
<code>`endpoint`</code>	Required. The server's https address. Calls may reach only this host
<code>`include`</code>	Optional comma-separated tool names to expose. Empty exposes all
<code>`risk.<tool>`</code>	Optional override: <code>`Read`</code> , <code>`Write`</code> , <code>`Send`</code> or <code>`Destructive`</code>

Gmail is the generic connector with Google OAuth and the scopes `gmail.readonly` and `gmail.compose`. It reads mail and creates drafts. Add `gmail.send` when connecting only if agents should send. The endpoint must be an MCP server that fronts Gmail. No address is built in.

An MCP server describes its own tools, so the connector decides risk itself, in this order: your `risk.<tool>` setting, then the definition's fixed risks, then the tool name. A name with a delete-like word (delete, remove, trash, purge, cancel and others) is Destructive. A name with a send-like word (send, post, publish, reply, forward) is Send. A server cannot talk a tool down from these. Only then are the server's `destructiveHint` and `readOnlyHint` used, and a name with a read-like word (get, list, search) is Read. Everything else is write. A misspelled `risk.` value stops the connection from starting.

If the server drops its session, the next check reconnects and the tools are fetched again.

Writing a connector

- Implement `ConnectorDefinition`: a `ConnectorDescriptor` (id, auth kind, allowed hosts, capabilities) and a factory.
- Implement `Connector` plus `IToolConnector`, `IEventSourceConnector` or both. Build tools with `ConnectorTools.Create`, which sets the prefixed name and the risk.
- Get credentials from `IConnectionContext.GetCredentialAsync` and the HTTP client from `CreateHttpClient`. Return failures as `ConnectorTools.Error(code, message)`.
- Register with `services.AddConnector(definition)`.
- Add a test class that derives from `ConnectorConformance` in `tests/AiAgentCanvas.Tests`. It checks tool naming, descriptions, declared risk, capability flags, check behavior against a failing service, double disposal and webhook rejection.

Limits to know

- **Approval list.** `Security:ApprovalRequiredTools` blocks a tool outright. It does not ask. `schedule_task`, `system_write_file`, `system_run_script` and `connect_mcp_server` are blocked by default and have no approval path. Connector tools use the approval requirement described above and are not in that list.
- **Runtime tools.** Tools that arrive while the host runs, from connectors and from servers connected with `connect_mcp_server`, are wrapped for governance and tracing and reach agents on their next call. An agent with a tool seed sees only the runtime tools its seed names.

- **Chat runs.** The ledger and the daily budgets cover unattended work. A chat session is not recorded and not limited by Agent :Budgets.
- **Approval channel.** Only the default interactive agent has an approval channel. A persona agent that calls a send tool does not execute it.

Typed Output, Vision, Orchestration and MCP Server

This guide covers four capabilities that widen what an agent can do and what can call it. Typed output makes a model answer in a shape a program can rely on. Vision lets a model look at images. Orchestration runs several agents together, with durable state and a human decision point. The MCP server lets outside clients call chosen tools.

Each is off by default behind a `Features: *` flag.

Flag	Adds	Needs
<code>`StructuredOutput`</code>	Answers checked against a JSON Schema and retried until they fit	none
<code>`Vision`</code>	Tools that describe an image or extract typed data from it	A vision-capable model. <code>`StructuredOutput`</code> adds the extraction tool.
<code>`AgentOrchestration`</code>	Group chat, handoff and Magentic runs that survive a restart	<code>`InterAgentCommunication`</code>
<code>`McpServer`</code>	A Model Context Protocol endpoint serving a chosen set of tools	Authentication on, or an explicit opt-out

Typed output

A model that is asked for JSON often returns JSON with a missing field, a string where a number belongs, or a sentence wrapped around the object. The structured responder expects that and does not trust the first answer.

For each request it does the following:

- Sends the schema twice: as the provider's native response format, and in the instruction text. A provider that ignores the format still follows the text.
- Parses the answer. A code fence or a sentence around the JSON is tolerated.
- Checks the parsed value against the schema.
- On a failure, sends the answer back with the specific errors (`$.age: expected integer but found string`) and asks again, up to the attempt limit.

The result says whether it succeeded, the value, the errors from the last attempt, how many attempts it took, and the raw text of the last answer. A failure comes back as a result and does not throw, so a caller does not mistake a default value for an answer.

The responder runs on the same chat pipeline as an agent, so each attempt is counted, priced and limited like any other model call.

From code. The shape can live in a class. `RespondAsync<T>` builds the schema from the type and returns a typed value.

```
public sealed record Weather(string City, double TempC);

var result = await responder.RespondAsync<Weather>(
    "Extract the weather.", "It is 21.5 degrees in Sydney.");

if (result.Success) Console.WriteLine(result.Value!.City);
```

From an agent. The tool `extract_structured` takes the text, an instruction, and either `schemaName` (a stored schema) or `schema` (a JSON Schema document as text). `list_schemas` shows the stored ones. The text may come from an outside party, so it can fill the fields but cannot change the shape or the instruction.

Stored schemas. Files named `*.schema.json` in `Agent:Structured:SchemaDirectory` are loaded at start. The file name without the suffix is the schema name. A file that is not valid JSON is skipped with a warning.

From HTTP. `POST /api/structured/extract` takes `instruction`, `input`, `schemaName` or `schema`, and `maxAttempts`. `GET /api/structured/schemas` lists the names. The endpoint key is `structured`.

What the validator checks. `type` (including a list of types), `properties`, `required`, `additionalProperties`, `items`, `enum`, `const`, `minimum`, `maximum`, `exclusiveMinimum`, `exclusiveMaximum`, `minLength`, `maxLength`, `pattern`, `minItems`, `maxItems`, `anyOf`, `oneOf`, `allOf`, and local `$ref` into `$defs` or `definitions`. It ignores other keywords, such as `format`, and does not treat them as failures. Errors are capped at 20 so a long bad array does not flood the retry message.

Setting (`Agent:Structured`)	Default	Meaning
`SchemaDirectory`	`schemas`	Folder of `*.schema.json` files
`MaxAttemptsCap`	5	The most attempts any request may use
`MaxInputChars`	100000	Longest input accepted
`MaxSchemaChars`	20000	Longest schema accepted from a caller

Vision

The vision tools pass images to a model that can read them. They need a vision-capable model behind the pipeline. A text-only model rejects the request, and the tool reports the model's error.

Tool	Does
`describe_image`	Answers a question about one or more images, or describes them

Tool	Does
<code>`extract_from_image`</code>	Reads an image into a JSON shape, checked and retried like typed output. Present only when <code>`StructuredOutput`</code> is on.

Both take sources: file paths or https addresses. Text in an image comes from whoever made it. The tools mark their result as untrusted content, and the model should treat it as data.

Where images may come from. The same rule the system tools follow applies: an empty list denies everything.

- `AllowedPaths` lists the folders files may be read from. A path outside them is refused, including a `..` path, a sibling folder whose name starts with an allowed one, and a link inside an allowed folder that points out of it.
- `AllowedUrlHosts` lists the hosts an image may be fetched from: an exact name or `*.example.com`. Only https is fetched. An address with a user name or password is refused.
- The fetch follows no redirects. Before connecting, it refuses private, loopback and link-local addresses unless `AllowPrivateNetworks` is true. The check runs on the address actually connected to, so a name that resolves to an internal address does not get through.

What is accepted. PNG, JPEG, GIF and WebP, decided by the file's first bytes and not by its name or the server's claim. SVG is refused because it can carry script. The size limit is enforced while reading, so a server that lies about its length is still stopped.

Setting (<code>`Agent:Vision`</code>)	Default
<code>`AllowedPaths`</code>	empty (no files)
<code>`AllowedUrlHosts`</code>	empty (no fetching)
<code>`AllowPrivateNetworks`</code>	false
<code>`MaxImageBytes`</code>	5242880
<code>`MaxImagesPerCall`</code>	4
<code>`FetchTimeoutSeconds`</code>	15

Images reach an agent through these tools. Attaching an image in the chat window is not supported.

Orchestration

An orchestration run puts several named agents on one task. Three kinds ship.

Kind	How it works
<code>`GroupChat`</code>	The agents take turns on one shared conversation, round robin, until the round limit

Kind	How it works
`Handoff`	A lead agent passes the conversation to a specialist by calling a handoff tool, and the specialist can pass it back. The model must support tool calls.
`Magentic`	A manager agent writes a plan, assigns work to the team, tracks progress and replans when the team stalls

The agents come from the agent registry, so each is a persona the host already knows. Each runs on the standard pipeline, which means a limit, a cost counter or a governance rule applies to a run as it does to a chat.

Durable by design. Each step is checkpointed to disk, and the run record lives in SQLite. A Magentic run asks for a person to approve its plan (`requireSignoff`, on by default). It then stops completely: nothing is running and nothing is waiting in memory. The record holds the plan and a notification goes out. When the person answers, possibly days later and after a restart, the host rebuilds the workflow from the record, restores the checkpoint and continues from the same point.

Status	Meaning
`Running`	Working now
`WaitingForInput`	Stopped at a checkpoint for a person's decision
`Completed`	Finished, with a result and a transcript
`Failed`	An agent or the model raised an error, or the run exceeded `RunTimeoutMinutes`
`Cancelled`	A person or an agent cancelled it
`Interrupted`	The process stopped mid-run. Resume it from the last checkpoint.

A person answers, an agent does not. The tools `start_orchestration`, `get_orchestration`, `list_orchestrations` and `cancel_orchestration` are available to agents. There is no tool to approve or revise a plan. An agent that could approve its own plan would make the sign-off meaningless. The answer goes through `POST /api/orchestrations/{id}/respond` (endpoint key `orchestrations`) or the **Orchestrations** tab, which shows the plan with **Approve**, **Ask for changes** (with feedback) and **Cancel run**.

Other endpoints: `GET /api/orchestrations` (filter with status), `GET /api/orchestrations/{id}`, `POST /api/orchestrations` to start, `POST /api/orchestrations/{id}/resume` for an interrupted run, and `POST /api/orchestrations/{id}/cancel`.

Recording. Each run appears in the run ledger with source `Orchestration`, so its cost and outcome sit with every other run. After a restart, runs left in `Running` are marked `Interrupted` and the log says how many. Finished runs and their checkpoints are deleted after `RetentionDays`. A run waiting for a person is kept however old it is.

Setting (`Agent:Orchestration`)	Default
`DatabasePath`	`orchestrations.db`
`CheckpointDirectory`	`orchestration-checkpoints`
`DefaultMaxRounds`	8
`MaxRoundsCap`	30
`MaxAgents`	8
`RunTimeoutMinutes`	15
`RetentionDays`	30

Limits.

- The only request a run can stop for is a Magentic plan review. A run built from code with a custom request port is not covered.
- The group chat manager is round robin. A manager that picks the next speaker with a model is not offered.
- A run executes inside one process. Checkpoints make a restart safe, and two hosts do not share a run.
- A handoff depends on the model calling the handoff tool. A model that ignores tools finishes with the lead.

MCP server

The MCP server lets another agent, an IDE or a workflow engine call tools on this host through the Model Context Protocol. It is the reverse of the MCP connector, which lets this host call someone else's tools.

Nothing is exposed until `Agent:McpServer:ExposedTools` names it. A name matches exactly, or by family with a trailing `*` (`rag_*`).

```
{
  "Features": { "McpServer": true },
  "Agent": {
    "McpServer": {
      "ExposedTools": ["extract_structured", "list_schemas", "rag_*"]
    }
  }
}
```

Safety rules.

- **Authentication.** The endpoint (default `/mcp`, endpoint key `mcp`) hands tools to whoever can reach it. The host refuses to start with `McpServer` on and `Authentication:Enabled` false. Set `Agent:McpServer:AllowUnauthenticated` to true only when the endpoint is reachable from a trusted network alone.
- **Approval.** A tool that needs a person's approval is not offered, even when named, because an MCP call has no one to ask. The log says so.

- **Governance and tracing.** Each exposed tool passes through the same governance wrapper and tracing as when an agent calls it. A governance rule that blocks a tool for an agent blocks it here.
- **Audit.** Each outside call is recorded in the run ledger as a run with source `External`, with the tool name, its arguments (cut to 500 characters) and a preview of the result.
- **Rate limit.** The endpoint carries the per-caller rate limit (`Security:RateLimitPerMinute`), like the other endpoints that spend model calls.
- **Result size.** A result longer than `MaxResultChars` is cut before it leaves, and the cut is marked.
- **Missing names.** A configured name the host does not register is logged and skipped. The rest still load.
- **Connector tools.** Tools that come from connections are not exposed, because they appear and disappear as connections change. Only tools registered at start are offered.

Setting (<code>`Agent:McpServer`</code>)	Default
<code>`Path`</code>	<code>`/mcp`</code>
<code>`ServerName`</code>	<code>`AiAgentCanvas`</code>
<code>`ExposedTools`</code>	empty (nothing)
<code>`AllowUnauthenticated`</code>	false
<code>`MaxResultChars`</code>	100000

Appendix A: User Guide

Getting Started

Prerequisites

- .NET 10 SDK
- Node.js 22 or later
- A model provider: an Azure OpenAI deployment, a Databricks serving endpoint, or a Snowflake Cortex account

Setup Steps

- Clone the repository.
- Build the backend. A solution build also copies the sample agent and market data plugins into the Host's `plugins/` folder:

```
dotnet build AiAgentCanvas.sln
```

- Configure `src/Host/AiAgentCanvas.Host/appsettings.Development.json` with your provider credentials. This example uses Azure AI Foundry:

```
{
  "AIFoundry": {
    "Endpoint": "https://YOUR-RESOURCE.openai.azure.com",
    "Key": "your-api-key",
    "DeploymentName": "gpt-4o",
    "UseAzureCredential": false
  }
}
```

To use Databricks or Snowflake, set `Provider` to `Databricks` or `Snowflake` and fill in that provider's section.

- Turn on the capabilities you want. All flags default to `false`, so a fresh start is a bare chat agent. This set matches the examples below:

```
{
  "Features": {
    "Personas": true,
    "Context": true,
    "Guardrails": true,
    "Skills": true,
    "SkillAuthoring": true,
    "Workflows": true,
    "InterAgentCommunication": true,
    "RunLedger": true
  },
  "Agents": { "FinancialAnalyst": { "Enabled": true } },
  "DataConnections": { "MarketData": { "Enabled": true } }
}
```

- Run the backend. It listens on `http://localhost:5149`:

```
dotnet run --project src/Host/AiAgentCanvas.Host
```

- In a separate terminal, start the frontend. The dev server forwards /api requests to the backend:

```
cd frontend
npm install
npm run dev
```

- Open `http://localhost:3000` in your browser.

Five Things to Try First

These assume the flags from step 4 above.

- **Ask a question.** Type any question in the chat box and watch the streaming response.
- **Create a persona.** Say "create a persona called Research Assistant that specializes in summarizing articles."
- **Create and run a skill.** Say "create a skill called Summarize that takes a URL and returns a summary," then "run skill Summarize."
- **Check a stock quote.** Say "get me a stock quote for MSFT" to exercise the market data tools.
- **Create a workflow.** Say "create a workflow called Morning Briefing that checks stocks then summarizes news."

After a few unattended runs, open the **Runs** tab to see what each one did and what it cost.

Configuration Reference

appsettings.json Structure

The base configuration file lives at `src/Host/AiAgentCanvas.Host/appsettings.json`. The template for new deployments is `appsettings.template.json` in the same folder. The main sections and keys:

Section	Key	Description
(root)	<code>`Provider`</code>	The LLM backend: <code>`AzureAIFoundry`</code> (default), <code>`Databricks`</code> or <code>`Snowflake`</code>
<code>`AIFoundry`</code>	<code>`Endpoint`, `Key`, `DeploymentName`</code>	Azure OpenAI endpoint, API key and chat deployment (for example <code>`gpt-4o`</code>)
<code>`AIFoundry`</code>	<code>`UseAzureCredential`</code>	Use <code>`DefaultAzureCredential`</code> instead of an API key
<code>`AIFoundry`</code>	<code>`EmbeddingDeploymentName`</code>	Embedding deployment. RAG needs it, together with the <code>`Rag`</code> flag
<code>`AIFoundry`</code>	<code>`EconomyDeploymentName`, `JudgeDeploymentName`</code>	A cheaper model for routing and summaries, and a separate model for the evaluation suite

Section	Key	Description
`Databricks`, `Snowflake`	`WorkspaceUrl` or `AccountUrl`, token, `ModelName`	The same settings for those providers, with `EmbeddingModelName`, `EconomyModelName` and `JudgeModelName`
`Security`	`PolicyPath`	Path to the governance policy YAML file
`Security`	`ApprovalRequiredTools`	Tool names that governance blocks. Defaults to `system_write_file`, `system_run_script`, `connect_mcp_server`, `schedule_task`
`Security`	`RateLimitPerMinute`	Requests per minute for each caller on the endpoints that spend model calls: AG-UI, A2A, structured output, orchestrations and the MCP server. Default 30. Zero or less turns it off.
`Authentication`	`Enabled`, `Schemes`, `AllowAnonymous`, `AllowedOrigins`, `ApiKey`, `JwtBearer`	Endpoint authentication. Off by default.
`Agent`	`ContextBudget`, `LoopGuard`, `Pricing`, `Reflection`, `ModelRouter`	The chat pipeline. See Platform Internals.
`Agent`	`Scheduler`, `EventTriggers`, `Budgets`, `RunLedger`, `Jobs`	Unattended work, run records and spend limits. See Operations and Connectors.
`Agent`	`Structured`, `Vision`, `Orchestration`, `McpServer`	The typed output, vision, orchestration and MCP server capabilities
`SystemTools`	`AllowedPaths`, `AllowedCommands`, `MaxFileSizeBytes`, `ScriptTimeoutSeconds`	Filesystem and shell allowlists. Empty means deny.
`Connections`	`PublicBaseUrl`, `KeyRingPath`, `OAuthApps`	The credential store and OAuth
`Connectors`	`ApprovalMode`, `CheckIntervalSeconds`	The connector host
`ChatHistory`	`ConnectionString`, `MaxMessages`	SQLite chat history

Section	Key	Description
`VectorStore`	`ConnectionString`	SQLite connection string for the RAG vector store
`ApplicationInsights`	`ConnectionString`	Azure Monitor connection string. Also exports the platform's own spans and metrics.

Feature Flags

Every capability in the platform can be individually enabled via the Features section. All flags default to false -- the platform is opt-in, so a fresh deployment registers nothing beyond the LLM provider and security until you turn a capability on. Set a flag to true to register that capability's services, tools, and endpoints.

```
{
  "Features": {
    "Personas": false,
    "Context": false,
    "Guardrails": false,
    "UserProfiles": false,
    "Entities": false,
    "Skills": false,
    "SkillRegistry": false,
    "SkillAuthoring": false,
    "Workflows": false,
    "Mcp": false,
    "SystemTools": false,
    "Notifications": false,
    "Scheduling": false,
    "Rag": false,
    "InterAgentCommunication": false,
    "EpisodicMemory": false,
    "AuditLog": false,
    "EventTriggers": false,
    "ComputerUse": false,
    "RunLedger": false,
    "Jobs": false,
    "Connections": false,
    "Connectors": false,
    "StructuredOutput": false,
    "Vision": false,
    "AgentOrchestration": false,
    "McpServer": false
  }
}
```

Flag	Description
`Personas`	Dynamic persona management. Each agent gets an identity, expertise area, tone, and behavioral rules injected into its system prompt. Personas can be created, swapped, or layered at runtime.

Flag	Description
`Context`	Persistent domain-specific context (facts, rules, reference material) loaded into the agent's system prompt on every turn.
`Guardrails`	Behavioral boundaries that constrain what the agent will and will not do. Guardrails are injected into the system prompt alongside the persona, enforcing policy at the reasoning level.
`UserProfiles`	User identity and preferences. The agent knows who it is talking to -- name, role, preferences, and permissions -- and adjusts its responses accordingly.
`Entities`	Long-term entity memory. Agents remember key entities (people, projects, systems, accounts) across conversations, stored in SQLite and recalled when relevant.
`Skills`	Named, multi-step procedures the agent can invoke by name. Skills are registered as tools and contain structured instructions the agent follows to execute complex workflows repeatably. Stored under `agent-data/skills`.
`SkillRegistry`	Discovery and listing of all registered skills. Enables agents to browse available skills and invoke them by name.
`SkillAuthoring`	Agents can create and edit skills at runtime through natural language instructions. New skills are persisted and immediately available.
`Workflows`	Orchestrated multi-step sequences involving tools, decisions, and checkpoints. Supports sequential, concurrent, and declarative (YAML) execution patterns.
`Mcp`	Model Context Protocol client connections. Registers the connection manager and its tools, and connects the servers named by `IMcpConnectionSeed` at startup. The `connect_mcp_server` tool is blocked by default.
`SystemTools`	Tools to read, write and list files and to run scripts, inside `SystemTools:AllowedPaths` and `AllowedCommands`. Governed by the same policy pipeline as custom tools. `system_write_file` and `system_run_script` are blocked by default.
`Notifications`	Agent-to-user notification delivery via SSE. Registers the notification store, tool provider, and the `/api/notifications` HTTP endpoints.

Flag	Description
`Scheduling`	Cron-based and one-time scheduled tasks that run an agent or a job. Persisted in SQLite and executed by a background service. <code>`schedule_task`</code> is blocked by default.
`Rag`	Retrieval-augmented generation backed by a vector store. Documents are chunked, embedded, and stored. At query time, the agent retrieves relevant chunks with hybrid search and reranking. Needs an embedding model on the Azure AI Foundry or Databricks provider.
`InterAgentCommunication`	Multi-agent coordination: agent registry, in-process handoff, background delegation, and asynchronous mailbox-based messaging between agents.
`EpisodicMemory`	Agents remember past goals, outcomes, and tool usage across sessions. Episodes are stored in SQLite with automatic relevance decay (5% every 6 hours, pruned below 1%). Recent episodes are injected into the system prompt as context.
`AuditLog`	Every model invocation, tool call, result, and error is recorded in a SQLite-backed audit trail. Sensitive parameters (keys, tokens, passwords) are automatically redacted. Agents can query their own history and retrieve aggregate statistics.
`EventTriggers`	Proactive agent engagement through scheduled (cron), file-watch, webhook and connector triggers. Events wait in a durable SQLite queue with deduplication, retry and a dead-letter state. Registers the trigger service, tool provider, and the <code>`api/triggers`</code> HTTP endpoints.
`ComputerUse`	Browser automation via headless Chromium (Playwright). Agents can navigate pages, click elements by coordinates or CSS selector, type text, take screenshots, and extract page content.
`RunLedger`	One SQLite record per scheduled, triggered, delegated and job run: input, output, tool calls, tokens, estimated cost and how it ended. Adds the <code>`api/runs`</code> endpoints and the Runs tab. Daily spend limits (<code>`Agent:Budgets`</code>) need it.
`Jobs`	Deterministic jobs that run without a model, started from a schedule, a trigger or an agent tool.

Flag	Description
<code>`Connections`</code>	Encrypted credential storage, the OAuth connect flow with PKCE, and token refresh. Adds the <code>`/api/connections`</code> endpoints and the Connections tab.
<code>`Connectors`</code>	Connectors to outside services (Twilio SMS, Gmail and other MCP servers). Needs <code>`Connections`</code> . Adds the connector host, <code>`/api/connectors/{connectionId}/webhook`</code> , and connector triggers.
<code>`StructuredOutput`</code>	Answers in a fixed JSON shape, checked against a schema and retried with the errors until they fit. Adds <code>`extract_structured`</code> , <code>`list_schemas`</code> and the <code>`/api/structured`</code> endpoints.
<code>`Vision`</code>	Tools that describe an image or extract typed data from it. Images come from allowlisted folders and https hosts. Needs a vision-capable model.
<code>`AgentOrchestration`</code>	Group chat, handoff and Magentic runs with checkpoints and a human sign-off step. Needs <code>`InterAgentCommunication`</code> . Adds the <code>`/api/orchestrations`</code> endpoints and the Orchestrations tab.
<code>`McpServer`</code>	Serves a chosen set of tools over the Model Context Protocol. Exposes nothing until <code>`Agent:McpServer:ExposedTools`</code> names it, and refuses to start with authentication off unless told otherwise.

The run ledger, spend limits, durable triggers, jobs, connections and connectors are described in Operations and Connectors. Typed output, vision, orchestration and the MCP server are described in Typed Output, Vision, Orchestration and MCP Server.

Service Modules (Agents and Data Connections)

Agent projects and data-connection projects use the `IServiceModule` interface instead of feature flags. Each module declares a configuration section name and self-registers its services. Host does not reference these projects at compile time at all: they're loaded at runtime from a `plugins/` folder next to the Host executable, one subfolder per plugin, via `System.Runtime.Loader.AssemblyLoadContext`.

`ServiceModuleExtensions.AddServiceModules` scans that folder, loads each plugin into its own isolated `PluginLoadContext`, and reflects over its types for `IServiceModule` implementations.

Modules are opt-in, like feature flags: a module with no configuration section, or with `Enabled` absent, stays disabled. Set `Enabled` to `true` in its configuration section to register it:

```
{
  "Agents": {
    "FinancialAnalyst": {
      "Enabled": true
    }
  }
}
```

```

    }
  },
  "DataConnections": {
    "MarketData": {
      "Enabled": true
    }
  }
}

```

To create a new module, implement `IServiceModule` in your agent or data-connection project:

```

public sealed class MyAgentModule : IServiceModule
{
    public string SectionName => "Agents:MyAgent";

    public void ConfigureServices(IServiceCollection services, IConfiguration configuration)
    {
        // Register persona seeds, tools, context, etc.
    }
}

```

Then wire up the project so its build output lands where the host will look for it: add `<GenerateDependencyFile>true</GenerateDependencyFile>` to its `.csproj`, plus a post-build target that copies the output into `$(HostPluginsDir)$(MSBuildProjectName)\` (copy the target verbatim from an existing plugin project, e.g. `Agent.FinancialAnalyst.csproj`). No `ProjectReference` from Host, no `Program.cs` edit, no line in Host's project file at all. The project only needs to be part of the solution so a normal build produces its output; Host finds it by folder, not by name.

Under the hood, `PluginLoadContext` resolves each plugin's own private dependencies from its folder via `AssemblyDependencyResolver`, but defers to whatever the host has already loaded for shared assemblies, above all `AiAgentCanvas.Abstractions`. Skipping that fallback would give the plugin's copy of `IServiceModule` a different runtime identity than the host's, and the type check that finds modules would silently fail.

Provider-Specific Configuration

Provider-specific settings are nested under their provider section. The Databricks Vector Search tool, for example, is configured under `Databricks:VectorSearch` and self-gates based on whether the required settings (workspace URL, token, index name) are present:

```

{
  "Databricks": {
    "WorkspaceUrl": "https://your-workspace.cloud.databricks.com",
    "ModelName": "databricks-meta-llama-3-3-70b-instruct",
    "VectorSearch": {
      "WorkspaceUrl": "https://your-workspace.cloud.databricks.com",
      "PersonalAccessToken": "dapi...",
      "IndexName": "main.default.docs_index"
    }
  }
}

```

Environment-Specific Overrides

ASP.NET Core loads configuration in layers. Place environment-specific values in:

- `appsettings.Development.json` -- local development settings (API keys, local endpoints).
- `appsettings.Production.json` -- production settings (real keys, production endpoints).

The environment-specific file merges on top of the base `appsettings.json`.

Environment Variable Overrides

Any configuration key can be overridden via environment variables using the double-underscore separator:

```
AIFoundry__Key=your-key-here
AIFoundry__Endpoint=https://prod.openai.azure.com
Security__RateLimitPerMinute=60
```

Frontend Configuration

The frontend needs no backend URL setting. In development, `npm run dev` forwards requests to `/api` to `http://localhost:5149`, the address in the Host's launch settings. To point the dev server somewhere else, change the rewrite in `frontend/next.config.ts`.

A production build is a static export. The Host serves it from `wwwroot`, so the browser and the API share one origin. The Docker image listens on port 5000.

Using the Chat Interface

UI Elements

- **Tabs** -- Chat, Runs (the run ledger with totals and detail), Orchestrations (multi-agent runs, and where a person answers a plan review) and Connections (connected accounts).
- **Streaming responses** -- text appears token by token as the model generates it.
- **Tool call indicators** -- a status bar shows which tools are running and when they complete.
- **State panel** -- displays structured data returned by tools (stock quotes, task lists, entity data).
- **Reasoning blocks** -- shows chain-of-thought when the model reasons through a problem, then fades after 5 seconds.
- **Interrupt approve/deny buttons** -- when the agent requests approval for a sensitive action, buttons appear to approve or deny.
- **Health status banner** -- displays connection and service health at the top of the page.
- **Notifications** -- server-sent events (SSE) push real-time updates for scheduled tasks, workflow completions, and errors.

Troubleshooting

Problem	Likely Cause	Solution
No response after sending a message	Backend is not running or the frontend cannot reach it	Verify the backend is running on <code>http://localhost:5149</code> and that the dev server's <code>/api</code> rewrite points to it

Problem	Likely Cause	Solution
Response starts then times out	Model deployment is overloaded or the API key has expired	Check the backend console for HTTP 429 or 401 errors; verify <code>`AIFoundry:Key`</code> and <code>`AIFoundry:Endpoint`</code>
Tool call shows "blocked by governance policy"	Governance policy denied the tool call	Review <code>`governance-policy.yaml`</code> for deny rules matching the tool; adjust the policy or use a different approach
State panel stays blank	The tool does not have a <code>`ToolStateMapping`</code> registered	Only tools with a registered <code>`ToolStateMapping`</code> emit state events; check if the tool is mapped in its service extensions
Health check failed banner appears	Backend health endpoint returned an error	Check backend logs for startup failures; verify database connectivity and API key validity
Requests return 429	The caller passed <code>`Security:RateLimitPerMinute`</code> on a model-spending endpoint	Wait for the <code>`Retry-After`</code> interval, or raise the limit. Callers who are not signed in share an allowance by address.
A tool call is refused and the tool is on the default list	<code>`Security:ApprovalRequiredTools`</code> blocks <code>`system_write_file`</code> , <code>`system_run_script`</code> , <code>`connect_mcp_server`</code> and <code>`schedule_task`</code>	Remove the name from the list only if you accept what the tool can do
API calls return 401	<code>`Authentication:Enabled`</code> is true and the request carries no credentials	Send the <code>`X-API-Key`</code> header, or a bearer token for the configured authority
The Host stops at startup with a message about <code>`Features:McpServer`</code>	The MCP server is on and authentication is off	Turn authentication on, or set <code>`Agent:McpServer:AllowUnauthenticated`</code> for a trusted network
A connection shows <code>`NeedsReauth`</code>	The provider refused the stored credential	Choose Reconnect on the Connections tab
An orchestration run is <code>`WaitingForInput`</code>	A Magentic plan needs a person's approval	Open the Orchestrations tab and approve, ask for changes, or cancel

Managing Agent Data

All agent data is managed through natural language commands in the chat. Each domain supports create, read, update, and delete operations.

Personas

- "Create a persona called **Technical Writer** that writes clear, concise documentation."
- "Switch to the **Technical Writer** persona."
- "List all personas."
- "Delete the **Technical Writer** persona."

Context

- "Save a fact: our fiscal year starts in July."
- "List all context entries."
- "Delete the context entry about fiscal year."

Entities

- "Save an entity for **Acme Corp** -- they are a partner company based in Seattle."
- "Search entities for **Acme**."
- "List all entities."
- "Delete the entity **Acme Corp**."

Guardrails

- "Create a guardrail called **No PII** with severity high: never include personal identifiable information in responses."
- "Toggle the **No PII** guardrail off."
- "List all guardrails."

User Profiles

- "Create a profile called **Developer** with role engineer, timezone US/Pacific."
- "Switch to the **Developer** profile."
- "List all profiles."

Workflows

- "Create a workflow called **Daily Report** that summarizes open tasks then drafts a status email."
- "Run the **Daily Report** workflow."
- "List all workflows."
- "Delete the **Daily Report** workflow."

Skills

- "Create a skill called **Translate** that translates text to a given language."
- "Run the **Translate** skill with target language French."

- "List all skills."

Scheduling

- "Schedule a task to run the **Daily Report** workflow every weekday at 9am."
- "List all scheduled tasks."
- "Cancel the **Daily Report** scheduled task."

schedule_task is on the default approval-required list, so governance blocks it until you remove it from Security:ApprovalRequiredTools.

Reference: AG-UI Protocol

The AG-UI (Agent-User Interface) protocol defines how the frontend communicates with the agent backend. AI Agent Canvas implements this protocol using server-sent events (SSE) over a single HTTP POST endpoint.

Endpoint

```
POST /api/copilotkit
Content-Type: application/json
```

The endpoint is mapped in `CoreServiceExtensions.UseAiAgentCanvas()` via `MapAGUIServer()` from the `Microsoft.Agents.AI.Hosting.AGUI.AspNetCore` package.

Request Format

```
{
  "threadId": "thread_abc123",
  "runId": "run_xyz789",
  "messages": [
    {
      "role": "user",
      "content": "What is the stock price of MSFT?"
    }
  ]
}
```

- `threadId` -- identifies the conversation thread. Reuse across messages to maintain history.
- `runId` -- unique identifier for this specific run.
- `messages` -- array of message objects with role and content.

Response Format

The response is an SSE stream:

```
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive
```

Each event is a JSON object on a `data:` line, with a `type` field indicating the event type.

Event Types

Text Streaming

Text messages are streamed in three phases:

```
data: {"type": "TEXT_MESSAGE_START", "messageId": "msg_001"}

data: {"type": "TEXT_MESSAGE_CONTENT", "messageId": "msg_001", "delta": "The current"}
```

```
data: {"type": "TEXT_MESSAGE_CONTENT", "messageId": "msg_001", "delta": " stock price of MSFT is $425.30."}
```

```
data: {"type": "TEXT_MESSAGE_END", "messageId": "msg_001"}
```

Tool Execution Lifecycle

Tool calls are bracketed by start and end events:

```
data: {"type": "TOOL_CALL_START", "toolCallId": "tc_001", "toolName": "stock_quote", "args": {"symbol": "MSFT"}}
```

```
data: {"type": "TOOL_CALL_END", "toolCallId": "tc_001", "result": {"symbol": "MSFT", "price": 425.30}}
```

State Updates

State events push structured data to the frontend state panel. Two behaviors exist:

Snapshot replaces the entire state panel content:

```
data: {"type": "STATE_SNAPSHOT", "snapshot": {"symbol": "MSFT", "price": 425.30, "change": "+1.2%"}}
```

Delta appends to or merges with the existing state panel content:

```
data: {"type": "STATE_DELTA", "delta": {"latestAlert": "Price crossed $425 threshold"}}
```

Agent Step Tracking

Steps track higher-level phases of agent execution:

```
data: {"type": "STEP_STARTED", "stepName": "Analyzing market data"}
```

```
data: {"type": "STEP_FINISHED", "stepName": "Analyzing market data"}
```

Chain-of-Thought Reasoning

Reasoning messages expose the model's thought process:

```
data: {"type": "REASONING_START"}
```

```
data: {"type": "REASONING_MESSAGE_CONTENT", "delta": "I need to look up the current price..."}
```

```
data: {"type": "REASONING_END"}
```

The frontend displays reasoning blocks temporarily and clears them after 5 seconds.

Run Lifecycle

```
data: {"type": "RUN_STARTED", "runId": "run_xyz789"}
```

```
data: {"type": "RUN_FINISHED", "runId": "run_xyz789", "outcome": "completed"}
```

When outcome is "interrupt", the frontend shows approve/deny buttons for the user to continue or cancel the interrupted action.

Error Reporting

```
data: {"type": "RUN_ERROR", "runId": "run_xyz789", "error": "Rate limit exceeded. Retry after 60 seconds."}
```

Why SSE over WebSockets

The AG-UI protocol uses server-sent events rather than WebSockets for several reasons:

- **Unidirectional fit** -- the agent streams output to the client; the client sends new requests as separate HTTP POSTs. This matches the SSE model naturally.
- **HTTP compatibility** -- SSE runs over standard HTTP, so it works with existing load balancers, reverse proxies, and CDNs without special configuration.
- **Auto-reconnection** -- the browser's EventSource API reconnects automatically on connection loss. The frontend fetch-based SSE reader handles this as well.
- **Simpler infrastructure** -- no WebSocket upgrade handshake, no connection state to manage on the server, no sticky sessions required.
- **Firewall friendly** -- SSE uses standard HTTP ports and protocols, avoiding firewall rules that block WebSocket upgrades.

Cancellation

The backend respects `HttpContext.RequestAborted` as a `CancellationToken`. When the client disconnects (navigates away, closes the tab, or explicitly aborts the fetch), the token fires and the agent stops processing. This prevents wasted compute on abandoned requests.

ToolStateMapping

The `ToolStateMapping` record and `ToolStateBehavior` enum control how tool results appear in the state panel:

```
public enum ToolStateBehavior
{
    Snapshot,    // Replaces the entire state panel
    Delta,       // Appends to existing state
}

public sealed record ToolStateMapping(string ToolName, ToolStateBehavior Behavior);
```

Tool state mappings are registered in each capability's service extensions. When the AG-UI stream encounters a result from a mapped tool, `AGUIStreamOptions.MapResult` routes it through either `ToStateSnapshot` or `ToStateDelta`, which emit the corresponding SSE event.

Registered mappings:

Tool Name	Behavior	Source
`list_workflows`	Snapshot	WorkflowServiceExtensions

Tool Name	Behavior	Source
`run_workflow`	Delta	WorkflowServiceExtensions
`run_sequential_workflow`	Delta	WorkflowServiceExtensions
`run_concurrent_workflow`	Delta	WorkflowServiceExtensions
`list_scheduled_tasks`	Snapshot	SchedulerServiceExtensions
`stock_quote`	Snapshot	MarketDataServiceExtensions
`stock_history`	Snapshot	MarketDataServiceExtensions
`edgar_company_facts`	Snapshot	MarketDataServiceExtensions
`list_available_agents`	Snapshot	OrchestrationServiceExtensions

Reference: Security and Governance

Governance Kernel

The `GovernanceKernel` (from the `Microsoft.AgentGovernance` NuGet package) is the central governance object. It is constructed from `GovernanceOptions` and exposes three components:

- **PolicyEngine** -- evaluates tool calls against the governance policy rules.
- **AuditEmitter** -- emits structured audit events for policy checks, violations, and blocked calls.
- **InjectionDetector** -- scans system instructions for prompt injection patterns.

Registration

Call `AddAiAgentCanvasSecurity()` in `Program.cs` before any other AI Agent Canvas service registration:

```
builder.Services.AddAiAgentCanvasSecurity(builder.Configuration);
```

This registers the following components as singletons:

Component	Purpose
<code>`GovernanceKernel`</code>	Central governance object
<code>`PolicyEngine`</code>	Policy evaluation (extracted from kernel)
<code>`AuditEmitter`</code>	Audit event emission (extracted from kernel)
<code>`GovernanceContextProvider`</code>	Scans system instructions for prompt injection
<code>`GovernedMcpGateway`</code>	Evaluates tool calls against MCP gateway rules
<code>`GovernanceToolWrapper`</code>	Wraps all <code>`AIFunction`</code> instances with governance checks
Rate limiter	Per-caller fixed-window rate limiter (ASP.NET built-in)

On the middleware side, `UseAiAgentCanvasSecurity()` applies security headers and subscribes to governance audit events for logging. `UseAiAgentCanvasRateLimiting()` enables the rate limiter and must run after authentication, so a limit can tell callers apart.

Tool-Call Governance Pipeline

Every tool call passes through a five-step governance pipeline:

1. Tool call received
|
2. Build `McpGatewayRequest` (`agentId`, `toolName`, `payload`)
|
3. Evaluate against policy rules via `GovernedMcpGateway`

4. Decision: Allow / Block / Audit
5. Execute the tool (if allowed) or return a JSON error (if blocked)

GovernedAIFunction

GovernedAIFunction extends DelegatingAIFunction (from Microsoft.Extensions.AI) and intercepts every tool call. It:

- Serializes the call arguments to JSON.
- Calls GovernedMcpGateway.Evaluate() to get an allow/block decision.
- Emits an audit event via AuditEmitter (either PolicyCheck or ToolCallBlocked).
- If blocked, returns a JSON error string: {"error": "Tool 'X' was blocked by governance policy.", "status": "..."}.
- If allowed, delegates to the inner function via base.InvokeCoreAsync().

GovernanceToolWrapper

GovernanceToolWrapper implements IToolGovernanceWrapper. AgentPipeline.WrapTools calls Wrap() on each AIFunction and wraps the result in TracedAIFunction. The default agent, persona agents, the tools served by the MCP server, and connector tools are wrapped this way.

Tools that arrive at runtime are wrapped as they are registered. That covers tools from a server connected with connect_mcp_server, which McpConnectionManager wraps, and tools from connectors, which the connector host wraps. The governance policy sees their calls, and the approval-required list applies to their names.

Approval-Required Tools

Security:ApprovalRequiredTools lists tool names that the governance gateway treats as needing approval. The list is a hard block: there is no approval prompt for these tools, and a call is refused until an operator removes the name from the list. The default is:

```
"Security": {
  "ApprovalRequiredTools": [
    "system_write_file",
    "system_run_script",
    "connect_mcp_server",
    "schedule_task"
  ]
}
```

Each entry must match the name a tool registers under. An entry that matches no tool protects nothing, so a typo raises no error.

Connector tools use a different mechanism. A connector tool with risk Send or Destructive is wrapped in ApprovalRequiredAIFunction, which asks a person before the call runs. The default interactive agent has an approval channel and shows the prompt in the chat. A persona agent or an unattended run has no channel, so it does not execute the call. Set Connectors:ApprovalMode to Audit to run those tools without asking. Calls are still traced and counted.

Policy Format

Governance policies are defined in YAML. The policy file path is set via `Security:PolicyPath` in configuration (default: `governance-policy.yaml`).

Structure

```
name: AiAgentCanvas-default
description: Default governance policy for AiAgentCanvas

rules:
  - name: block-dangerous-tools
    scope: tool_call
    action: deny
    condition:
      tool_name:
        in: [system_run_script]
    reason: "Shell execution requires explicit approval"

  - name: restrict-file-write-paths
    scope: tool_call
    action: deny
    condition:
      tool_name:
        equals: system_write_file
      path:
        matches: "^(/etc|/var|C:\\\\Windows|C:\\\\Program Files)"
    reason: "Writing to system directories is blocked"

  - name: block-private-mcp-endpoints
    scope: tool_call
    action: deny
    condition:
      tool_name:
        equals: connect_mcp_server
      endpoint:
        matches: "(localhost|127\\.0\\.0\\.1|169\\.254\\.169\\.254|10\\.|172\\.([6-9]|2[0-9]|3[01])\\.|192\\.168\\.)"
    reason: "MCP connections to private/internal addresses are blocked (SSRF protection)"

  - name: allow-all-other
    scope: tool_call
    action: allow
    condition: {}
```

Rule Fields

Field	Description
<code>`name`</code>	Unique identifier for the rule
<code>`scope`</code>	What the rule applies to (currently <code>`tool_call`</code>)
<code>`action`</code>	<code>`allow`</code> , <code>`deny`</code> , or <code>`audit`</code>
<code>`condition`</code>	Match criteria using operators (see below)

Field	Description
<code>`reason`</code>	Human-readable explanation shown when the rule fires

Condition Operators

Operator	Description	Example
<code>`equals`</code>	Exact string match	<code>`tool_name: { equals: system_write_file }`</code>
<code>`in`</code>	Matches any value in a list	<code>`tool_name: { in: [system_run_script, exec] }`</code>
<code>`matches`</code>	Regular expression match	<code>`path: { matches: "^/etc" }`</code>

Conflict Strategy

When multiple rules match a tool call, `ConflictResolutionStrategy.DenyOverrides` applies: any matching deny rule wins regardless of allow rules. This is the default and recommended setting.

Rate Limiting

The platform registers ASP.NET's fixed-window rate limiter with a policy named "agent". The Host attaches it to the endpoints that spend model calls: AG-UI, A2A, structured output, orchestrations and the MCP server. Health, run history, connection management, notifications, webhooks and the OAuth callback are not limited.

Each caller has an allowance of its own. The key is the authenticated identity when there is one, and the remote address otherwise. Callers who are not signed in share an allowance by address. Behind a reverse proxy, every request shares the proxy's address unless forwarded headers are configured. All callers who use one API key share an identity, set by `Authentication:ApiKey:PrincipalName`.

Parameter	Value
Window	1 minute
Permit limit	<code>`Security:RateLimitPerMinute`</code> for each caller (default 30). Zero or less turns the limit off.
Queue limit	0 (no queuing; excess requests are rejected immediately)

When the policy rejects a request, the server returns HTTP 429 with a `Retry-After` header in seconds and a JSON body:

```
{
  "error": "Rate limit exceeded. Try again later."
}
```

Security Headers

Three security headers are applied to HTTP responses through inline middleware in `UseAiAgentCanvasSecurity()`:

Header	Value	Purpose
<code>`X-Content-Type-Options`</code>	<code>`nosniff`</code>	Prevents MIME-type sniffing
<code>`X-Frame-Options`</code>	<code>`DENY`</code>	Prevents the page from being embedded in frames
<code>`Referrer-Policy`</code>	<code>`strict-origin-when-cross-origin`</code>	Limits referrer information sent to other origins

Authentication

Authentication is off by default. The Host logs a prominent warning at each start while it is off, because the endpoints it protects run shell commands, schedule unattended work and spend money.

`AiAgentCanvas.Authentication` picks schemes by name from `Authentication:Schemes`. Two ship: `ApiKey` for machine callers and `JwtBearer` for any OIDC authority, which covers Entra ID, Auth0, Okta and Keycloak.

```
{
  "Authentication": {
    "Enabled": true,
    "Schemes": ["ApiKey", "JwtBearer"],
    "AllowAnonymous": ["health"],
    "AllowedOrigins": ["https://app.example.com"],
    "ApiKey": { "HeaderName": "X-API-Key", "Keys": ["a-long-random-key"] },
    "JwtBearer": { "Authority": "https://login.example.com", "Audience": "agents" }
  }
}
```

Endpoints are protected with `RequireAgentAuthorization(auth, key)`, and `AllowAnonymous` lists the keys that stay open. The keys in use are `agui`, `a2a`, `devui`, `notifications`, `webhooks`, `health`, `runs`, `connections`, `connectors`, `structured`, `orchestrations` and `mcp`. API keys are compared in fixed time. Setting `AllowedOrigins` switches CORS from any origin to the named origins with credentials.

Two routes carry no endpoint authorization on purpose, because the caller is another service and sends none of our credentials:

- The **OAuth callback** (`/api/connections/oauth/callback`). The provider redirects the browser here. An encrypted, time-limited state value proves this host started the flow.
- The **connector webhook** (`/api/connectors/{connectionId}/webhook`). The connector checks the sender's own signature, and a request that fails is refused with 401 before anything is parsed.

Credentials and Connections

Connection secrets are encrypted with ASP.NET Core Data Protection before they reach the database. Metadata stays readable and secrets do not. The key ring lives in `Connections:KeyRingPath`, and on

Windows it is also wrapped with DPAPI. Whoever can read both the key ring and `connections.db` can read all stored secrets, so keep the key ring out of source control and out of the web root, and back up both together.

Secrets reach a connector only through its connection context. They do not appear in tool arguments, tool results, log lines or API responses. A connector's HTTP client allows only the hosts the connector names, over https (http for loopback only), follows no redirects, and retries only requests that are safe to repeat. The OAuth redirect address is built from `Connections:PublicBaseUrl` and not from the request, so a forged Host header cannot send an authorization code elsewhere.

Other Exposure Controls

Feature	Control
System tools	<code>`SystemTools:AllowedPaths`</code> and <code>`AllowedCommands`</code> both deny everything when empty. The Host defaults the path list to the <code>`agent-workspace`</code> folder and the command list to <code>`dotnet`</code> , <code>`git`</code> , <code>`npm`</code> and <code>`node`</code> . Commands run without a shell.
Vision	<code>`Agent:Vision:AllowedPaths`</code> and <code>`AllowedUrlHosts`</code> deny everything when empty. The fetch refuses private addresses at connect time.
MCP server	Exposes nothing until <code>`Agent:McpServer:ExposedTools`</code> names it. Leaves out tools that need approval. The Host refuses to start with it on and authentication off unless <code>`AllowUnauthenticated`</code> is set.
Orchestration	A person answers a plan review. No agent tool can.
Spend	<code>`Agent:LoopGuard:MaxRunCost`</code> caps a run, and <code>`Agent:Budgets`</code> caps daily spend per agent, per trigger and in total.

Production Checklist

Before deploying to production, verify each item:

- **Set real API keys.** Replace placeholder values in `AIFoundry:Endpoint` and `AIFoundry:Key` with production credentials, or enable `UseAzureCredential` for managed identity. Keep secrets in environment variables or a secret store.
- **Turn authentication on.** Set `Authentication:Enabled` to true, choose schemes, and list `AllowedOrigins`. Without it, endpoints are open to anyone who can reach the process.
- **Configure governance policies.** Review `governance-policy.yaml` and add deny rules for tools that should not run in production.

- **Review the approval-required list.** `schedule_task`, `system_write_file`, `system_run_script` and `connect_mcp_server` are blocked by default. Remove a name only when you accept what the tool can do.
- **Review the tool allowlists.** Audit `SystemTools:AllowedPaths`, `SystemTools:AllowedCommands`, and the vision folders and hosts. Remove anything the deployment does not need.
- **Set spend limits.** Fill in `Agent:Pricing` for the models in use, then set `Agent:LoopGuard:MaxRunCost` and `Agent:Budgets`. Budgets need the `RunLedger` flag.
- **Protect the credential store.** Put `Connections:KeyRingPath` somewhere private, back it up with `connections.db`, and set `Connections:PublicBaseUrl` to the public https address.
- **Configure HTTPS.** Run behind HTTPS termination, either a reverse proxy or Kestrel HTTPS.
- **Set the logging level.** Change `Logging:LogLevel:Default` to `Warning` or `Error` to reduce volume and avoid logging sensitive data.
- **Remove DevUI in production.** Wrap the `AddDevUI()` and `MapDevUI()` calls in an environment check so the development UI is not exposed.
- **Review MCP connection security.** The default policy blocks `connect_mcp_server` calls to private and internal addresses. Verify the rule is active. Prefer a connector for any server that needs credentials.

Reference: Platform Internals

Context Provider Chain

Context providers extend `AIContextProvider` (from `Microsoft.Agents.AI`) and run in DI registration order. Each provider's `ProvideAIContextAsync` method appends content to `AIContext.Instructions`, building up the system prompt that the agent receives.

Order	Provider	Source	What It Injects
1	<code>`GovernanceContextProvider`</code>	<code>AiAgentCanvas.Security</code>	Scans existing instructions for prompt injection and emits audit events if it finds any. It does not modify instructions.
2	<code>`SystemPromptProvider`</code>	<code>AiAgentCanvas.Orchestration</code>	The default system prompt
3	<code>`PersonaContextProvider`</code>	<code>AiAgentCanvas.AgentData.Personas</code>	The active persona's instructions
4	<code>`PersistentContextProvider`</code>	<code>AiAgentCanvas.AgentData.Context</code>	All saved context entries (facts, notes, preferences)
5	<code>`GuardrailContextProvider`</code>	<code>AiAgentCanvas.AgentData.Guardrails</code>	All enabled guardrail rules
6	<code>`UserProfileContextProvider`</code>	<code>AiAgentCanvas.AgentData.Profiles</code>	The active user profile (role, timezone, preferences)
7	<code>`EntityContextProvider`</code>	<code>AiAgentCanvas.AgentData.Entities</code>	The entity index listing known entities and their types
8	<code>`RagContextProvider`</code>	<code>AiAgentCanvas.Capabilities.Rag</code>	Relevant document chunks from hybrid search, as numbered citations. Present only when RAG is configured.
9	<code>`EpisodicMemoryContextProvider`</code>	<code>AiAgentCanvas.Capabilities.EpisodicMemory</code>	Recent episodes. Present when <code>`EpisodicMemory`</code> is on.

Order	Provider	Source	What It Injects
10	<code>`DynamicToolContextProvider`</code>	<code>AiAgentCanvas.Orchestration.Skills</code>	Adds tools registered at runtime, such as those from connectors and connected MCP servers, to the agent's tools. It is the only provider that changes tools and not instructions. The runtime gives each agent its own instance, filtered by that agent's tool seed.

The order follows the order of the registration calls in `Program.cs`: Security first, then the core runtime, then Personas, Context, Guardrails, UserProfiles and Entities, then RAG and episodic memory. A provider is present only when its flag is on. The dynamic tool provider is the exception: the runtime adds it to the default agent and to each persona agent as it builds them, and it does not come from the DI list.

Chat Pipeline

`AgentPipeline` builds the chat client chain and wraps tools. The default agent, each persona agent built by the registry, and the structured responder use it. Listed from the outside in:

Client	Present when	Notes
<code>`LoopGuardChatClient`</code>	<code>`Agent:LoopGuard:Enabled`</code> (default true)	Tool-round cap, token budget, optional <code>`MaxRunCost`</code> , repeat and stagnation detection
<code>`ReflectiveChatClient`</code>	<code>`Agent:Reflection:Enabled`</code>	Reflection prompt after consecutive tool rounds
<code>`AuditingChatClient`</code>	<code>`AuditLog`</code> flag	Records each model call
<code>`CostAwareModelRouter`</code>	<code>`Agent:ModelRouter:Enabled`</code>	Needs an economy model
<code>`ContextBudgetChatClient`</code>	<code>`Agent:ContextBudget:Enabled`</code> (default true)	Counts with the tokenizer named in <code>`Agent:TokenizerModel`</code>
<code>`CostTrackingChatClient`</code>	Always on	Prices from <code>`Agent:Pricing`</code> . An unpriced model reports tokens and no cost.
<code>`ToolDeduplicatingChatClient`</code>	Always on	Closest to the provider

`AgentPipeline.WrapTools` wraps each `AIFunction` in the governance wrapper when one is registered, then in `TracedAIFunction`. Tools that are not functions pass through unchanged.

Run Tracking

`RunTracking.RunAsync` gives a unit of work an ambient `AgentRunContext` and, when a ledger is registered, a run record. Scheduled agent tasks, trigger events, handoffs, jobs, orchestration runs and outside MCP calls use it. The context collects usage from the cost-tracking client and tool calls from the tracing wrapper, rolls a child's usage up to its parent, and feeds the per-run cost cap in the loop guard. The `BudgetGuard` reads the ledger to decide whether a new unattended run may start.

MarkdownFile Utility

`MarkdownFile` (`AiAgentCanvas.Abstractions` namespace) is a utility for reading and writing markdown files with YAML frontmatter. All agent data domains use it for persistence.

Methods

Method	Signature	Description
<code>`Parse`</code>	<code>`static MarkdownFile? Parse(string filePath)`</code>	Reads a file from disk and parses its frontmatter and body. Returns <code>`null`</code> if the file does not contain valid frontmatter delimiters (<code>`---`</code>).
<code>`ParseContent`</code>	<code>`static MarkdownFile? ParseContent(string content, string filePath)`</code>	Parses frontmatter and body from a string. Returns <code>`null`</code> if the content does not start with <code>`---`</code> or lacks a closing delimiter.
<code>`Write`</code>	<code>`static void Write(string filePath, Dictionary<string, string> frontmatter, string body)`</code>	Creates the parent directory if needed, serializes frontmatter as <code>`key: value`</code> lines between <code>`---`</code> delimiters, appends the body, and writes UTF-8.
<code>`LoadAll`</code>	<code>`static List<MarkdownFile> LoadAll(string directory, string pattern = "*.md")`</code>	Returns all parseable markdown files from a directory.
<code>`SanitizeFileName`</code>	<code>`static string SanitizeFileName(string name)`</code>	Lowercases the name and replaces spaces and underscores with hyphens.
<code>`Get`</code>	<code>`string Get(string key, string fallback = "")`</code>	Looks up a frontmatter key; returns <code>`fallback`</code> if missing.
<code>`GetBool`</code>	<code>`bool GetBool(string key, bool fallback = false)`</code>	Looks up a frontmatter key and parses it as a boolean.

Properties

Property	Type	Description
<code>`Frontmatter`</code>	<code>`Dictionary<string, string>`</code>	Parsed key-value pairs from the YAML frontmatter block
<code>`Body`</code>	<code>`string`</code>	Everything after the closing <code>`---`</code> delimiter
<code>`FilePath`</code>	<code>`string`</code>	The file path this instance was parsed from

Tool Design Guidelines

Follow these rules when building tools for AI Agent Canvas:

- **Use descriptive names matching the domain.** Tool names should read naturally: `stock_quote`, `list_personas`, `run_workflow`. Avoid generic names like `execute` or `process`.
- **Write clear [Description] attributes.** The description is included in the model's prompt. Be specific about what the tool does, what it returns, and when to use it.
- **Return structured JSON.** Tools should return JSON-serializable objects. The AG-UI protocol and state panel depend on structured output.
- **Handle errors gracefully.** Return error information as part of the result rather than throwing exceptions. The model can interpret and relay error messages to the user.
- **Keep tools focused.** Each tool should do one job. Prefer two small tools over one tool with a `mode` parameter.
- **Accept `CancellationToken`.** Pass the token through to all async operations. This allows the system to cancel work when the user disconnects.
- **Document parameters with [Description].** Every parameter should have a description attribute so the model knows what values to pass.

Seed Interface Reference

Seeds provide default data for each agent domain. They are resolved from DI at startup. If the corresponding file does not already exist on disk, the seed data is persisted. Seeds never overwrite files that a user has manually edited.

Interface	Concrete Type	Properties	Persistence Path
<code>`IPersonaSeed`</code>	<code>`PersonaSeed`</code>	<code>`Name`</code> , <code>`Description`</code> , <code>`Instructions`</code>	<code>./agent-data/orchestrator/agent/personas/`</code>
<code>`IContextSeed`</code>	<code>`ContextSeed`</code>	<code>`Topic`</code> , <code>`Type`</code> , <code>`Tags`</code> , <code>`Content`</code>	<code>./agent-data/orchestrator/agent/context/`</code>
<code>`IWorkflowSeed`</code>	<code>`WorkflowSeed`</code>	<code>`Name`</code> , <code>`Description`</code> , <code>`Tags`</code> , <code>`Content`</code>	<code>./agent-data/orchestrator/agent/workflows/`</code>

Interface	Concrete Type	Properties	Persistence Path
<code>IEntitySeed`</code>	<code>EntitySeed`</code>	<code>`Name`, `Type`, `Tags`, `Content`</code>	<code>./agent-data/orchestrator/agent/entities/`</code>
<code>IGuardrailSeed`</code>	<code>GuardrailSeed`</code>	<code>`Name`, `Severity`, `Enabled`, `Rule`</code>	<code>./agent-data/orchestrator/agent/guardrails/`</code>
<code>ISkillSeed`</code>	<code>SkillSeed`</code>	<code>`Name`, `Description`, `PromptTemplate`</code>	<code>./agent-data/skills/`</code>
<code>IUserProfileSeed`</code>	<code>UserProfileSeed`</code>	<code>`Name`, `Role`, `Timezone`, `Content`</code>	<code>./agent-data/orchestrator/agent/profiles/`</code>
<code>IMcpConnectionSeed`</code>	<code>McpConnectionSeed`</code>	<code>`Name`, `Endpoint`, `Transport`, `BearerToken`, `ApiKey`, `ExpectedIssuer`, `AdditionalHeaders`</code>	(via MCP connection manager)
<code>IAgentToolsSeed`</code>	<code>AgentToolsSeed`</code>	<code>`AgentName`, `ToolNames` (list)</code>	(in-memory tool assignment)
<code>IGoalSeed`</code>	<code>GoalSeed`</code>	<code>`Name`, `Description`, `Priority`, `AcceptanceCriteria`, `AssignedAgent`, `Content`</code>	<code>./agent-data/orchestrator/agent/goals/`</code>

Each domain also supports user-created data that persists under the user/ subtree (e.g., `./agent-data/orchestrator/user/personas/`) and shared data under `./agent-data/shared/`.

Seed Behavior

- At startup, the DI container resolves all registered `ISeed` implementations for each domain.
- For each seed, the store checks if a file with the sanitized name already exists.
- If the file does not exist, the seed data is written using `MarkdownFile.Write()`.
- If the file exists, the seed is skipped -- manual edits are preserved.

RAG Pipeline Internals

The RAG (Retrieval-Augmented Generation) pipeline is enabled when the `Rag` feature flag is on and the active provider has an embedding model: `AIFoundry:EmbeddingDeploymentName` for Azure AI Foundry, or `Databricks:EmbeddingModelName` for Databricks. It adds relevant document context to the agent's system prompt before each response.

DocumentChunker

The `DocumentChunker` class splits text into overlapping chunks for ingestion.

Parameter	Default	Description
`ChunkSize`	512	Maximum characters per chunk
`ChunkOverlap`	64	Characters of overlap carried from the previous chunk

The chunking algorithm:

- Split text by double-newline (paragraph boundaries).
- Accumulate paragraphs into a buffer until ChunkSize would be exceeded.
- Flush the buffer as a DocumentChunk, carrying the last ChunkOverlap characters forward.
- If a single paragraph exceeds ChunkSize, split it further by sentence endings (. , ! , ?).
- Discard chunks shorter than 20 characters.

Hybrid Search

Hybrid search combines vector similarity with keyword matching. The weights are configurable via RagSearchOptions:

Component	Weight	Method
Vector search	0.7 (default)	Cosine similarity against stored embeddings
Keyword search	0.3 (default)	SQLite FTS5 BM25 ranking

The IHybridSearchable interface:

```
public interface IHybridSearchable
{
    IEnumerable<(DocumentRecord Record, double? Score)> HybridSearchAsync(
        ReadOnlyMemory<float> queryEmbedding,
        int top,
        RagSearchOptions? options = null,
        CancellationToken ct = default);
}
```

RagSearchOptions fields: SourceFilter, TagFilter, KeywordQuery, KeywordWeight (default 0.3f), VectorWeight (default 0.7f).

The final score for each document is: (VectorWeight * vectorScore) + (KeywordWeight * keywordScore).

FTS5 Schema

The SQLite FTS5 virtual table is created alongside the main documents table:

```
CREATE VIRTUAL TABLE IF NOT EXISTS [{collection}_fts]
USING fts5(id UNINDEXED, text, content=[{collection}], content_rowid=rowid)
```

- id is stored but not indexed (marked UNINDEXED).

- text is the searchable column.
- The FTS table is a content-sync table that mirrors the main documents table.

Keyword scores are normalized as $1.0 / (1.0 + |\text{rank}|)$ where rank is the FTS5 BM25 score.

LLM Reranking

After hybrid search retrieves the top candidates, an LLM reranker narrows the results:

- Retrieve retrieveK candidates (default: 10) via hybrid search.
- Truncate each candidate's text to 300 characters.
- Send a prompt to the LLM asking it to return a JSON array of chunk indices ranked by relevance (e.g., [2, 0, 4, 1, 3]).
- Parse the JSON array, deduplicate indices, and take the top topK (default: 3).
- On any parse or LLM error, fall back to the original ordering truncated to topK.

LLM reranking parameters: Temperature = 0, MaxOutputTokens = 100.

DocumentRecord

```
public sealed class DocumentRecord
{
    [VectorStoreKey]
    public string Id { get; set; }

    [VectorStoreData]
    public string Text { get; set; }

    [VectorStoreData]
    public string? Source { get; set; }

    [VectorStoreData]
    public string? Tags { get; set; }

    [VectorStoreData]
    public string? MetadataJson { get; set; }

    [VectorStoreVector(1536, DistanceFunction = CosineSimilarity)]
    public ReadOnlyMemory<float> Embedding { get; set; }
}
```

The embedding dimension is 1536 (compatible with OpenAI text-embedding-ada-002 and similar models). The VectorStoreVector attribute specifies cosine similarity as the distance function.